

Antonio Ferrão Neto

THE METAPHOR OF THE BUBBLELAND

A PROTOSCIENTIFIC APPROACH

Appendices: Methods and Formalism



Volume III

 **ImpNet**

THE METAPHOR OF THE BUBBLELAND

A Protoscientific Approach

Antonio Ferrão Neto

VOLUME III

Appendices: Methods and Formalism



© 2026 Antonio Ferrão Neto

All rights reserved. No part of this work may be reproduced, stored in a retrieval system or transmitted, in any form or by any means — electronic, mechanical, photocopying, recording or otherwise —, without the prior written permission of the author, save for the statutory exceptions for free use, among them quotation in academic work with due credit, under art. 46 of Brazilian Law 9,610/98.

Exception — openly licensed material. The **figures**, the **code listings** and the **cover images of all three volumes** are not covered by the reservation above: they are released under a **Creative Commons Attribution 4.0 International** license (CC BY 4.0) and may be reproduced, adapted and used freely, including commercially, provided the source is credited — *The Metaphor of the Bubbleland*, by Antonio Ferrão Neto. The figures and listings can be downloaded from the *QR codes* in the captions or directly at
<<https://www.impnet.com.br/bolholandia/en/>>.

Typeset with LaTeX using the ABNT template.

Dados Internacionais de Catalogação na Publicação (CIP)
(Câmara Brasileira do Livro, SP, Brasil)

Ferrão Neto, Antonio

The Metaphor of the Bubbleland : a protoscientific approach - Appendices: Methods and Formalism / Antonio Ferrão Neto ; [tradução do autor]. -- São Bernardo do Campo, SP : ImpNet, 2026. -- (The metaphor of the Bubbleland ; 3)

Título original: A metáfora da Bolholândia.

Bibliografia.

ISBN 978-65-977067-4-7

1. Biologia molecular 2. Computação 3. Equações diferenciais 4. Física 5. Mecânica quântica
I. Título. II. Série.

26-403410.0

CDD-530.12

Índices para catálogo sistemático:

1. Mecânica quântica : Física 530.12

Eliete Marques da Silva - Bibliotecária - CRB-8/9380

APPENDIX **X**

Brief introduction to the Successive Zooms Method (SZM)

*Nature uses only the longest threads to weave her patterns,
so that each small piece of her fabric reveals the
organization of the entire tapestry.*

— RICHARD P. FEYNMAN (FEYNMAN, 1965, p. 34)

IN KEEPING with the metaphorical spirit of this work, this Appendix X occupies a singular position in the book.

If the preceding chapters and appendices describe the landscape — conceptual mountains, seas of uncertainty, mapped regions and others shrouded in fog —, the

SZM is the navigation map itself (Figure 1). But the ‘X’ on this map does not mark a result or a final theory: it marks a procedure. The SZM assumes that certain systems do not admit a global description or a closed analytical solution, and proposes a local, iterative, and hierarchical exploration, in which each level of resolution reveals new structures. The treasure, if it exists, emerges from the process — not from the map.



Figure 1 – The treasure map as an epistemological metaphor: the “X” does not indicate a ready-made object, but the point where descriptive cartography ends and the method of exploration begins.



This epistemological stance echoes contemporary discussions about the limits of computation. Syropoulos (2009), for example, argues that the Universe is, in essence, analog and non-granular, harboring physically uncomputable phenomena — which makes illusory the pretension of describing it entirely by means of a conventional digital program. It is not for the SZM to settle such fundamental questions; but it is on this horizon — that of systems that resist a global, closed description — that the method inscribes itself, proposing a local, continuous, and iterative exploration,

more attuned to the analog nature of the world than to the strictly discrete paradigm of the Turing Machine.

Since the dawn of mathematics, humanity already intuited methods that, in a sense, anticipated the logic of the SZM. A notable example is the Babylonian clay tablet known as YBC 7289, shown in Figure 2, dated to around 1800–1600 BC¹.



Figure 2 – Tablet YBC 7289 (c. 1800–1600 BC), with a precise sexagesimal approximation of the square root of 2. It is now in the Yale Babylonian Collection (FOWLER; ROBSON, 1998).



Source: photograph by A. Urcia, Yale Peabody Museum of Natural History (public domain, CCO), via Wikimedia Commons.

The Babylonians used a sexagesimal number system (base 60). On the diagonal of the square on the YBC 7289 tablet, they wrote the following sequence of numbers: 1, 24, 51, 10. When we translate this into our modern decimal system, the precision of their calculation becomes evident:

$$1 + \frac{24}{60} + \frac{51}{60^2} + \frac{10}{60^3}$$

$$= 1 + 0.4 + 0.014166 \dots + 0.000046 \dots = \mathbf{1.41421296 \dots}$$

¹ YBC 7289 is a clay tablet from the Old Babylonian period that presents the square root of 2 in sexagesimal notation with precision equivalent to six decimal places — considered the greatest computational precision recorded in the ancient world. Dated to between approximately 1800 BC and 1600 BC, the tablet is believed to have been produced by a student in southern Mesopotamia.

The true value of $\sqrt{2}$ is 1.41421356 . . . This means that the Babylonian calculation was correct to six significant digits.

Although its goal was not to seek “arbitrary precision” as in the SZM, the underlying logic — working with partial results to arrive, step by step, at more refined values — shows that this way of thinking is ancestral. The great difference is that, for millennia, technological limitations prevented the extensive application of this approach. Today, with modern electronics, digital control, and artificial intelligence, historical barriers have dissolved, allowing the full potential of this idea to be explored.

1. Status of the SZM: concept, simulation, and hardware

Why is there (not yet) a physical SZM machine?

The SZM is a large-scale engineering project, which requires:

- the design and fabrication of boards with SZMDs (digitally configurable analog blocks);
- integration with FPGAs or microcontrollers for real-time reconfiguration;
- the development of firmware and software for control, calibration, and monitoring;
- laboratory infrastructure with precision instrumentation (oscilloscopes, spectrum analyzers, reference sources);
- financial investment, a multidisciplinary team, and a development time estimated in years.

None of these resources were available for this work. Yet the absence of physical hardware **does not invalidate the method** — as the historical precedent of artificial neural networks demonstrates: they existed as a concept validated in simulation for decades before hardware allowed their materialization (see Section X.1.3).

The SZM finds itself in an analogous situation. The principle was formulated, the architecture was described (Section X.1), and two proofs of concept in software were carried out:

1. **Deterministic proof of concept** (Section X.2.4, Listing AG.7): the program `szm_device.py` demonstrates the incremental extraction of digits by successive zooms, with shielding of the target value — the device reveals only blocks of two digits per interaction, without ever exposing the complete number.
2. **Monte Carlo simulation with a noise model** (Section X.4, Table 2): three independent noisy devices read each block of digits; a *unanimity-of-three* monitor detects disagreements and orders a **reprocess until success**. The study spans from a *terrible* device (2 digits per read, 10 % noise on the last digit) to a **cheap, realistic 3½-digit** device. The central result is **feasibility**: the value is recovered in about 99.99 % of the sessions across the whole tested range — and the rare residual failure is the *unanimous error* (the three devices agree on the *same* incorrect value), which no vote among the three can detect, but which Richardson extrapolation (Section X.6), beyond the scope of this proof of concept, would reduce only in the step-dependent part — a common-mode bias independent of step size survives it. At the recommended operating point — the cheap 3½-digit device — recovery is practically perfect and reprocessing negligible. Table 2 summarizes representative scenarios; the complete

continued →

1. Status of the SZM: concept, simulation, and hardware *(continued)*

grid is on the electronic page of Listing AG.10, reachable through the QR code in the listing's caption.

Just as neural networks went from “theoretical architecture” to “transformative technology” when hardware matured, the SZM is formulated, simulated, and documented — awaiting the moment when engineering resources allow its physical materialization.

Currently, the overwhelming majority of researchers seek to solve their complex problems on digital supercomputers with state-of-the-art specifications. But often these methods fail in the face of the so-called “stubborn problems” (see Appendix V, Vol. II). The SZM offers an alternative and ingenious architecture for dealing with such challenges — such as difficult differential equations — that elude traditional approaches. The method provides arbitrary precision from a mechanism with a minimum of only two decimal digits of precision guaranteed per stage.

The aim here is not to describe the SZM exhaustively, since it constitutes a large and costly engineering project that lies beyond the scope of this book. The objective here is to present only a general idea of the method.

X.1 Description of the method

The idea of iterative refinement is ancestral — as the tablet YBC 7289 attests (Figure 2). However, what the SZM proposes as novelty is not refinement in itself, but rather the systematic concatenation of blocks of digits obtained by a low-precision analog device, digitally controlled, to build a result of arbitrary precision.

In this concatenation, each block of two digits is extracted by a cycle of zoom + translation over the analog residue. Such an approach has no precedent in the computational architectures of the twentieth century, where analog and digital computation operated as separate, mutually exclusive paradigms.

Suppose a function f represents some numerical value whose mantissa (without the decimal point) is composed of the digits:

$$f = d_0 d_1 d_2 d_3 d_4 d_5 d_6 d_7 d_8 d_9 d_{10} \dots$$

Assume that the analog system can supply only the first two digits with precision. The first extracted block would be:

$$b_0 = d_0 d_1$$

With this, the amplified residue is defined:

$$r_0 = 100(f - b_0)$$

The first two digits of r_0 provide the second block:

$$b_1 = d_2 d_3$$

The operation repeats over the new residue:

$$r_1 = 100(r_0 - b_1) = d_4 d_5 d_6 \dots$$

And so on. At each iteration, two new significant digits are recovered. In general:

$$b_k = \text{first two digits of } r_{k-1}, \quad r_k = 100(r_{k-1} - b_k), \quad r_{-1} \equiv f \quad (\text{X.1})$$

After j stages, we obtain the sequence:

$$b_0 \| b_1 \| b_2 \| \dots \| b_j \Rightarrow d_0 d_1 d_2 \dots d_{2j+1}$$

Algorithm 1 Successive Zooms Method — Extraction of digit pairs

Require: Normalized mantissa $m \in [1, 10)$, precision p

Ensure: List of blocks B , list of residues R , list of accumulators A

```

1: threshold  $\leftarrow 10^{-(p-1)}$ 
2: maxSteps  $\leftarrow \lfloor (p+2)/2 \rfloor + 2$ 
3:  $B \leftarrow []$ ;  $R \leftarrow []$ ;  $A \leftarrow []$ ; accum  $\leftarrow ""$ 
4: for  $i = 0$  to maxSteps  $- 1$  do
5:   if  $m \leq 0$  then
6:     break
7:   end if
8:    $b_i \leftarrow \lfloor 10 m \rfloor$ 
9:   next  $\leftarrow b_i / 10$ 
10:   $r_i \leftarrow 100 (m - \textit{next})$ 
11:  if  $|r_i| < \textit{threshold}$  then
12:     $r_i \leftarrow 0$ 
13:  end if
14:   $m \leftarrow r_i$ 
15:  block  $\leftarrow \text{FORMAT}(b_i, 2 \text{ digits})$ 
16:  accum  $\leftarrow \textit{accum} \| \textit{block}$ 
17:   $B.\text{append}(\textit{block})$ 
18:   $R.\text{append}(r_i)$ 
19:   $A.\text{append}(\textit{accum})$ 
20: end for
21: return  $B, R, A$ 

```

► Extract block b_i : the two significant digits

► Compute residue r_i by zoom + translation

► Numerical noise cleanup

The method therefore transforms a low-precision analog machine into a system of arbitrary precision — provided that errors are controlled at each step.

Algorithm 1 seeks to convey the idea of the method, but it must be stressed that the implementation of the algorithm must be done essentially in **hardware**, since the core of the method is analog computation.

2. Why is there no quantitative comparison between the SZM and other numerical methods?

A direct quantitative comparison between the SZM and conventional numerical methods (such as Runge–Kutta or arbitrary-precision libraries like MPFR) is not possible at this stage, since the SZM is a hardware proposal that has not yet been materialized. It is worth noting, however, that arbitrary-precision libraries in software do not solve the problem of numerical stiffness: in strongly stiff problems, the integration step can become so small that dozens of digits would be needed merely to represent the increment — and, even so, the calculation does not advance, because the method remains trapped in tiny steps. Moreover, arbitrary-precision arithmetic in software does not use the processor's native operations, making each elementary operation orders of magnitude slower. In the SZM, precision is built by the concatenation of analog blocks — which decouples the final precision from the order of the numerical method employed and from the machine's arithmetic. A long discussion of these numerical difficulties was developed in Appendix V (Vol. II).

3. Dispelling the “floor of the physical quantity” objection

Where does precision live?

The intuitive objection — or the *Taylor objection*, since it reasons about the terms of the series expansion. On first encountering the SZM, it is almost inevitable to reason as follows: *“If the analog device is noisy and has low precision, how can it deliver arbitrary precision? At some point the zoom will hit the background noise of the physical quantity and there will be nothing left to extract.”* This objection seems solid — but it starts from a mistaken premise about *where* precision is generated.

The misconception. The intuition fails because it looks for precision *inside* the physical quantity, as if each zoom were merely “looking more closely at the same noisy measurement.” Under this reading, there would indeed be an unreachable “floor”: the point at which the component's noise swallows the signal. But the SZM does **not** work this way.

continued →

3. Dispelling the “floor of the physical quantity” objection (*continued*)

Where precision really lives. The precision of the SZM does not reside in the analog component. It inhabits two distinct places:

1. **In the mathematical formulation.** The coefficients of the equation (e.g. a, b, c in $ax^2 + bx + c = 0$) are exact numbers, defined digitally. They have whatever precision one wants, because they are *numbers*, not measurements.
2. **In the scale transformation.** Each zoom *rescales the problem* and submits it again to the SZMD. The fine structure of the result is not *extracted* from the physical quantity — it is *generated* by the mathematical recursion (exact coefficients + rescaling). The physical part acts only as a coarse ruler, reapplied at each level.

The decisive consequence. There is no “floor” to reach, because the fine structure was never in the physical quantity. At each level, the SZMD need only deliver 2 or 3 significant digits — exactly the range in which it operates comfortably. The component always works *within* its comfort zone, reading the most significant digit of the current block (the protected one), never the marginal digit drowned in noise. The total precision is *sliced* into blocks, and each block is resolved in isolation; the device is never asked for more than it can give.

Illustration ($\sqrt{2}$). Consider solving $x^2 - 2 = 0$, whose root $\sqrt{2} = 1.41421356 \dots$ is irrational. The SZM proceeds:

- **Zoom 0:** obtains “1,” and fixes the 1;
- **Zoom 1:** rescales the problem around 1; the SZMD reads “4” (safe zone of 2–3 digits);
- **Zoom 2:** rescales again and reads “1”, then “4”, “2”, “1” . . .

At each level, the equation is exact and the device operates far from its noise limit. The irrationality of the root imposes no “floor”: the recursion can proceed indefinitely, because the source of precision is the rescaled mathematics, not the analog ruler.

Why trained intuition goes wrong here. It is worth stating precisely why Taylor’s objection, correct as it is within its own domain, does not reach the SZM. Let λ be the horizontal scale factor, μ the vertical one, and ℓ_k the affine part already known at stage k ; the transformation used is $g(t) = \mu [f(a_k + \lambda t) - \ell_k(t)]$, with the subtraction performed *before* the vertical rescaling. Without that subtraction one would have $g' = \mu \lambda f'$ and $g'' = \mu \lambda^2 f''$: requiring the curvature to be visible, $g'' \sim O(1)$, would force $\mu \lambda^2 \sim 1$, and the slope would blow up, $g' \sim 1/\lambda \sim 10^k$, far beyond the range of any instrument. The objection would be right — for a magnifying glass, which enlarges without touching the object. With the subtraction, however, no slope remains to be amplified: it was removed, not scaled. What is left is the sagitta, and $\mu \sim 10^k$ brings it back to order unity.

There is a second inversion here, worth recording because it runs against a rule taught

continued →

3. Dispelling the “floor of the physical quantity” objection *(continued)*

early. Numerical analysis warns against cancelling nearly equal quantities, since in floating point this destroys the significant digits of the difference (WILKINSON, 1994; HIGHAM, 2002). The SZM does precisely the opposite, and can do so for an exact reason: the cancellation takes place in integer arithmetic, where there is no digit to lose. What would be noise in floating point is, here, the signal. The requirement is the same one that underpins iterative refinement, in which the residual must be computed to higher precision than the solution being corrected; the difference is that in the SZM the residual so obtained does not correct a solution — it defines the next problem.

The technique used in the SZM should not be confused with multiple-precision algorithms, such as “Bignum arithmetic,” which amounts to using several memory words to represent a single number. Although the SZM also combines digits to obtain “arbitrary precision,” it differs fundamentally from purely digital algorithms such as MPFR (“Multiple Precision Floating-Point Reliable”) and GMP (“GNU Multiple Precision Arithmetic Library”).

4. Dispelling the “long run” objection

Where does scale live?

The intuitive objection — in two voices. The previous box dispelled the objection about *precision*; this one is its twin on the axis of *time*, and it arrives through two different readers. The numerical analyst reasons thus: “*Rescaling the problem does not change its Jacobian. If the equations have components varying over picoseconds and others over seconds, that stiffness is still there at every zoom — and stiffness is what makes integrators fail.*” The analog-computing engineer reasons differently but arrives at the same place: “*To span twelve orders of magnitude in time within one run, the integrator would have to run for hours; amplifier drift and capacitor leakage eat the digits long before that, and the fast component lies above the op-amp’s bandwidth. The scales do not fit in the window.*”

The misconception. Both voices assume the same thing: that the problem is handed to the circuit *whole*, in a single long run — as in the classical analog computer, which wired up the equation and let it run, and as in the digital *solver*, which advances step by step from start to finish. Under that reading there would indeed be a “floor” of scale: the point where drift swallows the signal or the bandwidth cannot follow the fast component. But the MZS does **not** work that way. What undergoes the successive zooms is the *mathematical problem*; the circuit is reconfigured at each zoom to compute

continued →

4. Dispelling the “long run” objection (*continued*)

one window, and that window only.

Where scale really lives. Scale does not reside in the analog component. It inhabits two places:

1. **In the windowing.** The trajectory is traversed in windows of *constant width, comfortable for measurement*, not in one long run. One second of circuit time may represent picoseconds of the problem in one window and years in the next; what changes between them is the time-scale factor, not the circuit. Each DMZS is restarted at every window and “tells its story” delivering the same two or three digits as always.
2. **In the rescaling.** Amplitude and time scaling — done by hand in classical analog computers, potentiometer by potentiometer — is here the job of the DMZS control circuit, and can be automated (including by a neural network trained to choose the scale factors of each window). The analog component *always* works in its comfort zone; the scale is decided outside it.

The decisive consequence. There is no “floor” of stiffness to reach, because the scales were never in the circuit. What stiffness costs in the MZS is not feasibility: it is the *number of windows* — and hence total computation time, which depends on the amplifier, the D/A and A/D converters and the control circuit. It is the same inversion as in the previous box: there, precision lived in the rescaled mathematics and not in the ruler; here, scale lives in the windowed problem and not in the circuit’s clock.

Illustration (the Robertson problem). The three-reaction system of Appendix Z (Section Z.4.1.1) is the standard stiffness test: one rate constant of 0.04 and another of $3 \cdot 10^7$ in the same equations. In a single run, either the fast component does not fit the bandwidth or the slow one does not fit the duration. In the MZS, the fast transient is traversed in short windows and the slow regime in long ones — each rescaled, each read by the same DMZS with the same two digits. Stiffness becomes a count of windows, not an obstacle.

Why trained intuition fails here. The numerical analyst is right on one point, and it is worth stating it precisely. Let $y' = f(t, y)$ be the equation, $\tilde{y}_k$ the part of the solution already known up to level k , and $r = 10^k(y - \tilde{y}_k)$ the residual; within a window, the residual obeys

$$r' = 10^k [f(t, \tilde{y}_k + 10^{-k}r) - \tilde{y}'_k],$$

whose Jacobian with respect to r is the same as that of f with respect to y : the zoom does *not* change the eigenvalues. The analyst concludes from this that stiffness survives — and it does, as *scale*. What does not survive is the *step condition*: in the circuit there is no integration step, hence no step to violate, and the only demand the eigenvalues

continued →

4. Dispelling the “long run” objection (*continued*)

impose is the window’s scale factor, which item 2 above absorbs. The objection is true about the problem and empty about the method. The analog engineer, in turn, starts from a false premise — the long run — and that objection dissolves without remainder.

What remains open. Within a window, the DMZS must receive the already-known part $\tilde{y}_k(t)$ as a signal, so as to integrate only the residual. There are two ways to serve it: play back, through a D/A converter, a digital table of $\tilde{y}_k(t)$ at the window’s time resolution, or generate it by a coarser analog run in parallel. In either case the precision of the reinjection is not that of the converter: the same residual trick applies to the restart itself, which is why each DMZS can be restarted and go on telling its story with the same two digits. The choice between the two forms, and the cost of each, are left to the hardware design.

Although the idea of extracting incremental information through iterative cycles superficially recalls the principle of sigma-delta converters, the SZM differs fundamentally: in the SZM, each cycle extracts significant digits of the mantissa by zoom + translation, whereas in sigma-delta, oversampling and noise-shaping operate on the representation of the signal in the time domain. The two concepts share the principle of trading speed for resolution, but the mechanics are distinct.

X.1.1 Requirements

- A parallel monitoring and validation system, analogous to the role of error-correcting enzymes in DNA replication, ensuring that each stage maintains the required precision².
- An analog computation mechanism that guarantees error bounded to less than one hundredth per stage.
- High execution speed, since analog computation occurs on the order of nanoseconds.

The method is, therefore, a proposal for a hybrid analog-digital computational architecture.

X.1.2 Why did the SZM not emerge in the twentieth century?

Although conceptually simple, the SZM requires billions of dynamic, sequential configurations in the analog circuits. No human being would be able to keep up with this process on the analog computers of the time. Moreover:

² This is one more similarity that could be considered in Box 5.

- High-precision electronic components were lacking.
- There was no support from digital systems for control and automation.
- Modern artificial intelligence techniques were lacking.

5. Curiosity — Life is a hybrid analog-digital system

The most interesting mathematics involves the interplay between the continuous and the discrete.

— PAUL SCHUPP

Life itself can be regarded as a **hybrid analog-digital** system. Continuous processes, such as chemical gradients and electrical signals (**analog**), coexist with the discrete encoding of genetic information in DNA, RNA, and proteins (**digital**). This integration between analog and digital is also present in the SZM, which combines continuous and discrete processing to solve complex problems.

X.1.3 The SZM and technological inevitability

The SZM is a clear example of how certain scientific concepts can exist long before their practical materialization. Although the conceptual principle of the SZM — the iterative, multiscale analysis of complex phenomena — may find parallels in ancient methods of decomposition and approximation, its **computational feasibility** only became plausible in the present, thanks to the convergence of advances in hardware, algorithms, and data availability. Analog computation, which historically faced limitations of precision and programmability (MACLENNAN, 2007; MACLENNAN, 2004), now finds a new context thanks to the convergence of advances in hardware, algorithms, and data availability (ULMANN, 2022).

This pattern, in which an idea precedes its practical execution by decades or even centuries, is not unprecedented in the history of science and technology. **Artificial neural networks**, for example, have their origins in the work of McCulloch and Pitts (MCCULLOCH; PITTS, 1943), who in 1943 mathematically modeled an artificial neuron. Later, Rosenblatt's *Perceptron* (ROSENBLATT, 1958) and the developments of Werbos (WERBOS, 1974) and Rumelhart, Hinton, and Williams (RUMELHART; HINTON; WILLIAMS, 1986) established the foundations of supervised learning by backpropagation. Yet it was only from the mid-2000s onward — with the massive use of GPUs, the explosion of large databases, and the introduction of more efficient algorithms, such as those of Hinton, Osindero, and Teh (HINTON; OSINDERO; TEH, 2006), culminating in the consolidation presented by LeCun, Bengio, and

Hinton (LECUN; BENGIO; HINTON, 2015) — that *deep learning* became a practical and transformative tool. Throughout that interval, neural networks existed as a *concept validated in simulation, awaiting hardware*.

In an analogous way, the technology of **unmanned aerial vehicles** (*drones*) well illustrates how a concept can remain technically possible, but economically or logistically infeasible, for decades. Prototyped since the First World War, as in the case of the *Kettering Bug* (EVERETT, 2015), and developed in military contexts throughout the twentieth century, drones only became accessible and multifunctional in the twenty-first century. This was made possible thanks to the miniaturization of inertial sensors (IMUs) (YAZDI; AYAZI; NAJAFI, 1998), advances in lithium-ion batteries (NAGAURA; TOZAWA, 1990; TARASCON; ARMAND, 2001), improvements in high-efficiency *brushless* motors (YEDAMALE, 2003), the integration of low-cost GPS (KAPLAN; HEGARTY, 2017), and the development of robust embedded systems, such as the PX4 framework (MEIER; HONEGGER; POLLEFEYS, 2015). This technological convergence reduced costs, increased reliability, and multiplied applications in mapping, agriculture, public safety, and audiovisual production.

The case of the SZM, therefore, is not an isolated exception, but part of a recurring pattern: concepts that, even when formulated much earlier, await technological maturity to become realizable. **Technological inevitability** occurs when multiple fields converge, creating the historical moment in which an old idea ceases to be mere theory and becomes practice. And, just as happened with these technologies, the SZM is not only possible — it is, at some point in our future, inevitable.

X.1.4 The new technological context

Today, the SZM becomes feasible thanks to:

- GPGPUs and FPGAs with precise digital control.
- Fast analog-to-digital converters (AD/DA).
- Components such as digipots and precise, fast operational amplifiers.
- Neural networks capable of adjusting and validating the circuits thousands of times per second.

Thus, the digital computer — which in the twentieth century was the executioner of analog computation — can now become its main ally (TSIVIDIS, 2018). This possibility is not merely speculative: Cowan, Melville and Tsvividis (2006) demonstrated an analog VLSI co-processor integrated with a digital computer, and Huang et al. (2017) extended the hybrid approach to the solution of nonlinear partial differential equations. With the support of artificial intelligence and the new digital control

electronics, the Successive Zooms Method reopens the path to analog computation of arbitrary precision. This approach can solve problems hitherto considered intractable and, in this sense, proposes a rupture in the traditional way of thinking about scientific computing. A proposal that, if confirmed, may have impacts comparable to the greatest milestones of computational science.

In metaphorical terms, using artificial intelligence in the Successive Zooms Method is equivalent to using lubricating oil on the gears of mechanical machines. It is possible to build machines whose gears work without lubrication, but they will be noisy, will have a much shorter useful life, and will consume far more energy. Similarly, it is also possible to create SZM machines without artificial intelligence, but they will be less consistent, which results in reprocessing and loss of performance.

X.2 Definition of an SZM device (SZMD)

In the context of the SZM, we define an **SZM Device** (abbreviated as **SZMD**) as any electronic apparatus capable of performing some kind of computation or data processing, operating on one or more input values and producing one or more output values.

X.2.1 Composition and configurability of the SZMD

A **SZMD** can be:

- i) an elementary block (e.g., adder, subtractor, multiplier, divider, root extractor, exponentiator, logarithm extractor, integrator, differentiator, amplifier, buffer, inverter, A/D, D/A, voltage reference); or
- ii) a *superblock* composed by the interconnection of multiple elementary SZMDs.

Figure 3 presents some examples of analog blocks, while Figure 4 shows an SZMD composed of a *Sample & Hold* analog block and an *Integrator* analog block.

All signal timing is controlled by the computer, which commands the microcontroller/FPGA; the latter configures the SZMDs, receives their results, and returns them to the computer. Many functions can occur in parallel.

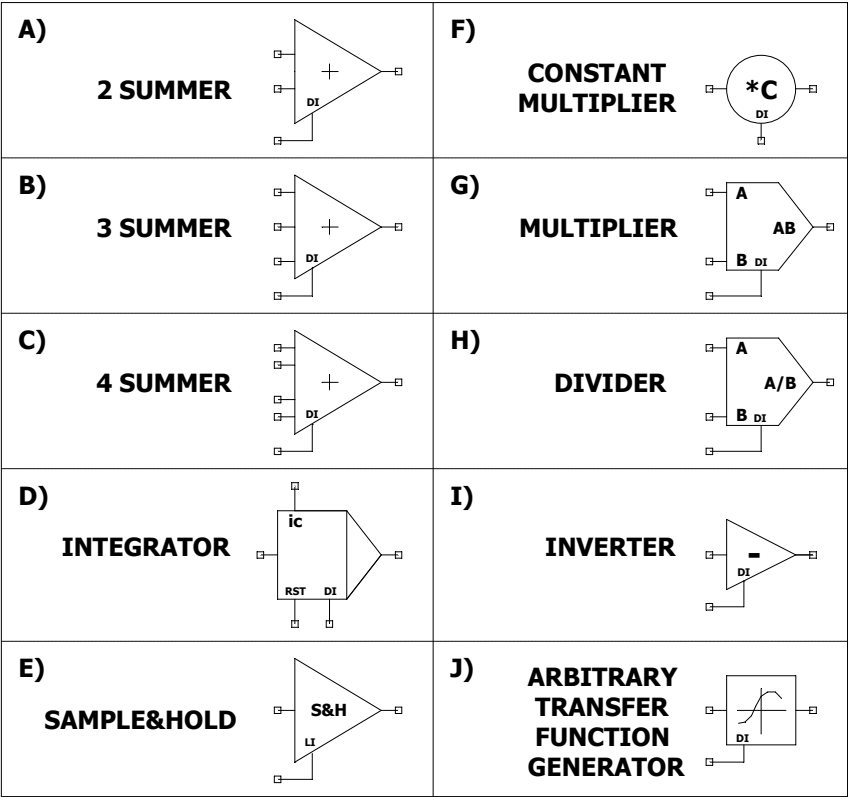


Figure 3 – **Just a few examples of digitally controlled analog blocks.** In addition to their standard inputs and outputs, they have a digital interface (DI, for “Digital Interface”), capable of configuring the analog block. For example, the DI can change the resistance value of a digital potentiometer, which sets the gain of the inverting amplifier (“INVERTER”). The same applies to the other blocks. In the case of the integrator (“INTEGRATOR”), the DI interface can command the switching of an analog switch between capacitors, changing the RC constant of the integrator.



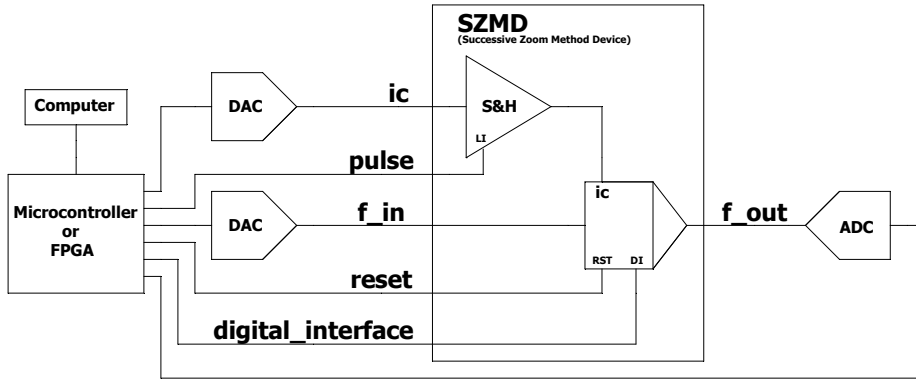


Figure 4 – **Example of an INTEGRATOR SZMD.** A computer — with or without the aid of GPGPUs — commands hardware implemented in a microcontroller or FPGA, which in turn physically configures the integrator block through the digital interface (“digital_interface”). The integrator can be initialized via a *reset* pin, discharging the capacitor. A digital-to-analog converter (DAC) digitally receives the value of the initial conditions and converts it to an analog value. The microcontroller or FPGA sends a pulse so that the *Sample & Hold* block samples and holds this value, which is applied to the “IC” pin of the INTEGRATOR block, thus defining the initial conditions of the analog integration. A second DAC receives digital data of the function to be integrated (“*f_in*”) and converts it into the integrator’s analog input signal. The integrated signal (“*f_out*”) is captured by an analog-to-digital converter (ADC), reconverting the integrated value into a sequence of digital values, which return to the microcontroller/FPGA and, subsequently, to the computer. Several of the lines in the diagram represent buses.



The precision of the final result of the SZM is not limited by the precision of the individual SZMD; the latter affects only the number of retries needed per block and the verification overhead, but not the total number of extractable digits.

X.2.2 Analog computation of the twenty-first century

Today's analog and digital components are very different from those available up to the 1960s, when analog computers began to be replaced by digital ones.

Classical analog computers used operational amplifiers now considered obsolete. Today, there are millions of different models — produced by various manufacturers — covering a huge range of electrical characteristics, noise, precision, consumption, and operating frequency.

Potentiometers, formerly exclusively mechanical (some coupled to servomotors for automatic adjustments), were replaced by digital potentiometers, which can be updated electronically through digital interfaces at great speed.

Analog-to-digital (ADC) and digital-to-analog (DAC) converters, which were once assembled with discrete components, are today widely available as monolithic integrated circuits, with numerous different resolutions, speeds, and architectures. Voltage references, essential to these converters, are also easily found as dedicated ICs, and, in many modern models, already come embedded within the converters themselves.

Analog multipliers — which enable not only multiplication, but also division, square-root extraction, and other algebraic operations — appeared relatively late. When the first commercial integrated circuit for analog multiplication, the AD534 (Analog Devices, Inc., 2013), was launched in the early 1970s, the coffin lid of the great analog computers was already closing.

Analog switches and selectors (multiplexers/demultiplexers), originally mechanical and manually operated, are today available in the form of digitally controlled integrated circuits³.

There are also, ready for use, monolithic *sample-and-hold* integrated circuits. Thus, there is no longer any need to design such blocks from discrete transistors: the LF398, from Texas Instruments (Texas Instruments, 2000), is a popular example that integrates the FET switch and two operational amplifiers in an 8-pin package. There are also higher-performance models, such as the AD783 from Analog Devices, which incorporates an internal capacitor and guarantees typical acquisition times of the order of 0.4 μ s for a precision of 0.01 % (HOROWITZ; HILL, 2015, p. 256).

Instead of resurrecting standalone analog computers, the twenty-first century incorporated analog computation as a subsystem of mixed-signal systems, in which

³ Examples: 74HC4051, ADG608, ADG609, ADG714, CD4051, CD4052, CD4053, CD4066, and CD74HC4067 (HOROWITZ; HILL, 2015)

specialized analog circuits perform continuous operations — such as integration, multiplication, weighted summation, and detection — while digital processors carry out logical control and high-level programming. This new paradigm appears in sigma-delta converters, adaptive-control ASICs, neuromorphic devices, processors with field-programmable analog arrays (FPAA), and highly integrated vision and RF systems. In this way, one can say that analog computation “has returned,” but not as an independent machine, rather as a functional block embedded within digital architectures that exploit the best of both worlds.

X.2.2.1 FPGAs, microcontrollers, and GPGPUs

Today, we have at our disposal a variety of digital hardware resources that did not exist until the 1970s, such as FPGAs (*Field Programmable Gate Arrays*), the numerous types of microcontrollers, and GPGPUs (*General Purpose Graphics Processing Units*).

FPGAs and microcontrollers are extremely useful for the rapid configuration of SZMDs, something essential, since each zoom in the SZM requires a specific parametrization. Moreover, as we will see in Section X.8.4, in addition to the process of successive zooms, it is possible to multiply SZMDs by reusing them in the formation of increasingly complex circuits, capable of representing much more elaborate coupled systems of differential equations. This possibility of reconfiguring the SZMDs countless times is precisely one of the factors that make the SZM feasible today.

Another resource enabled by modern technology, nonexistent before the advent of GPUs, concerns artificial neural networks (see Section X.8.1). The process of automatically carrying out the zooms can be guided by conventional deterministic computation combined with digital circuits, together with artificial intelligence algorithms based on neural networks. For this, the neural network is initially trained with known data, so that it can subsequently be executed millions of times over unknown data. The use of GPGPUs — in place of traditional GPUs — proves particularly convenient in this context, owing to the need for tight integration between the digital boards and the hardware responsible for configuring the SZMDs.

X.2.3 Mathematical abstraction of the functional model and ports

We represent an SZMD by a mapping

$$\mathbf{y} = F(\mathbf{x}; \mathbf{u}, \theta),$$

where $\mathbf{x}$ are *inputs*, $\mathbf{y}$ are *outputs*, $\mathbf{u}$ are *external controls* (configuration), and θ are *internal parameters* (components, gains, offsets, etc.). The SZMD can be *active* (it has a voltage/current supply and can provide power), and its function F can be changed at runtime via $\mathbf{u}$, changing the hardware “procedure” (an analogy with the function/procedure of programming languages, but implemented physically).

X.2.3.1 Network composition

SZMDs can be interconnected to form networks: (i) *series/cascade* $F_2 \circ F_1$; (ii) *parallel/sum* $F_{\parallel} = F_1 \oplus F_2$; (iii) *feedback* $F_{fb} = \mathcal{L}(F, H)$. The analogy with *two-port networks* (ABCD, Z , Y) is useful for port analysis, but here it extends to *active blocks configurable* by $\mathbf{u}$, not restricted to bilateral linear relations.

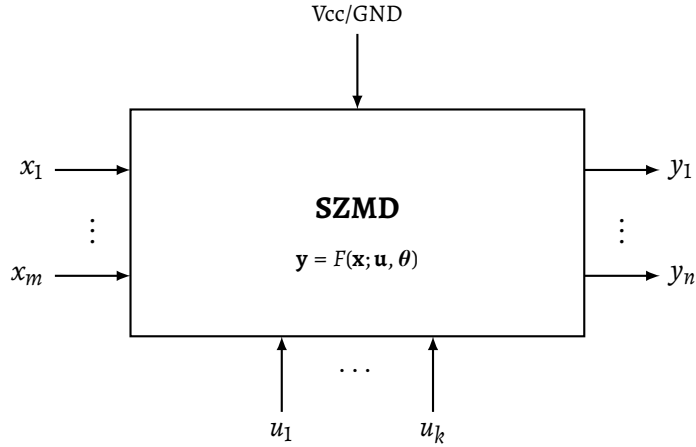


Figure 5 – Generic block of an SZMD with inputs $\mathbf{x}$, outputs $\mathbf{y}$, controls $\mathbf{u}$, and power supply.



X.2.3.2 Practical remarks

- Configurability via $\mathbf{u}$ allows the same SZMD to perform different functions (e.g., integrator $\leftrightarrow$ differentiator), with the possibility of adjusting configurable parameters θ_{cfg} (such as gains via *digipots*) without modifying the structural parameters θ_{fix} .
- For modular analysis, it is useful to publish the SZMD’s “port sheet” — as in the examples in Table 1 and Figure 6 — including: the domains of $\mathbf{x}$ and $\mathbf{u}$, the ranges of $\mathbf{y}$, the precision P (Equation X.5), and dynamic limitations.

- In networks, the effective precision results from the combination of the precisions of each SZMD and of the interconnection topologies (cascade, parallel, feedback).

Table 1 – SZMD Port Sheet

Identification	SZMD–001 (example)
Input domain x	$x \in [x_{\min}, x_{\max}]$ (unit); $ x \leq \dot{x}_{\max}$
Control domain u	$u \in [u_{\min}, u_{\max}]$; modes: INT, DIFF, HOLD
Output range y	$y \in [y_{\min}, y_{\max}]$ (unit); soft saturation beyond y_{sat}
Precision P	$P = 1 - \left[\alpha E_r + \beta \frac{\sigma}{ V_{\text{true}} } \right]$ (X.5); effective digits: d_{eff}
Dynamic limitations	Bandwidth BW, settling time t_s , jitter $\mathcal{J}$, max. update rate $f_{\max}$
Environmental conditions	Temperature $[T_{\min}, T_{\max}]$, tolerable EMI, vibration
Remarks	Configurable parameters θ_{cfg} (e.g., gains via <i>digipots</i>); structural parameters θ_{fix}

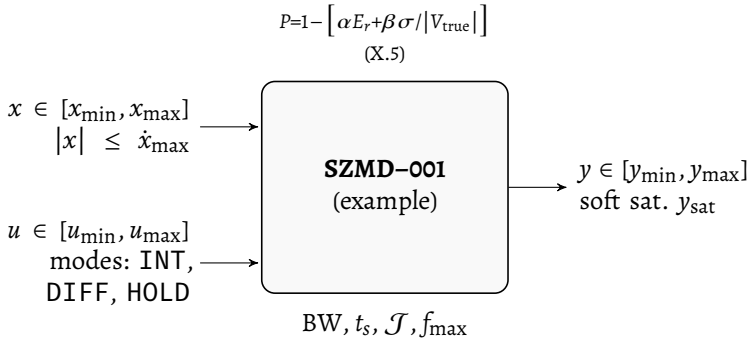


Figure 6 – Block diagram of the SZMD, with inputs, outputs, and main parameters.



Every SZMD has an intrinsic **precision**, which can be limited or affected by various factors, such as:

- deviations inherent to the apparatus itself;
- the presence of noise, whether internal or external;
- degradation resulting from the useful life and wear of the components;
- environmental conditions, such as temperature, humidity, mechanical vibrations, and electromagnetic interference.

Although, in principle, an apparatus capable of performing computations may be mechanical, hydraulic⁴, chemical, biological⁵ or based on other physical principles, in the SZM we will consider exclusively **electronic apparatuses**⁶, so as to maintain the pragmatic and technological focus of the method.

X.2.3.3 Precision of an SZM device

In this work, the **precision** of an SZMD will be understood as the capacity to produce results that approach the true or ideal expected value, considering the operating conditions. This precision is influenced by:

1. **Resolution** (Δ_{res}): the smallest increment detectable or representable by the SZMD.
2. **Accuracy**: the degree of closeness between the measured value and the true value.
3. **Repeatability**: the capacity to produce the same result under the same operating conditions.

In the SZM, precision will be treated as a quantifiable parameter that incorporates both systematic errors (consistent deviations) and random errors (unpredictable fluctuations), allowing the modeling and continuous monitoring of the SZMD's performance.

Mathematical formulation of precision

Let:

- V_{true} — the true or ideal value.
- $V_{\text{meas},i}$ — the value measured by the SZMD at the i -th observation.
- N — the number of repeated measurements.

⁴ Mechanical and hydraulic analog computers were used until the mid-twentieth century, such as the *Differential Analyzer* (mechanical), developed by Vannevar Bush at MIT in the 1930s, and the *MONIAC* (hydraulic), created by Bill Phillips in 1949 to simulate the workings of a national economy, by means of water flows in tanks and tubes. A general description of this equipment is found in (TRUITT; ROGERS, 1960).

⁵ Biological computation — that is, systems that use molecules or biological structures (for example, DNA, proteins, or cellular circuits) to perform operations — is under development and described in recent studies (BENENSON, 2012). Chemical computation, based on self-organized chemical reactions (such as reaction-diffusion), has been used to process information, including in Belousov–Zhabotinsky-type systems (PARRILLA-GUTIERREZ et al., 2020).

⁶ The electronic analog computer (GPAC — *General-Purpose Analog Computer*) was widely used until the 1960s, when the digital computer gradually began to replace it in many technical and scientific applications (Elsevier B.V., 2025).

We define:

$$\text{Absolute Error: } E_a = |V_{\text{meas}} - V_{\text{true}}|, \quad (\text{X.2})$$

$$\text{Relative Error: } E_r = \frac{E_a}{|V_{\text{true}}|}, \quad (\text{X.3})$$

$$\text{Variance: } \sigma^2 = \frac{1}{N-1} \sum_{i=1}^N (V_{\text{meas},i} - \bar{V}_{\text{meas}})^2, \quad (\text{X.4})$$

where:

$$\bar{V}_{\text{meas}} = \frac{1}{N} \sum_{i=1}^N V_{\text{meas},i}$$

is the mean of the measurements.

The **effective precision** P of the SZMD will be expressed as:

$$P = 1 - \left[\alpha \cdot E_r + \beta \cdot \frac{\sigma}{|V_{\text{true}}|} \right], \quad (\text{X.5})$$

where α and β are weighting coefficients that adjust the relative importance between the systematic error (E_r) and the random error (σ) according to the SZM application.

X.2.4 POC — description of the program **szm_device.py**

The program in Listing AG.7 presents a “proof of concept” (POC) of the principle of the idea of the SZM implemented in Python. The objective is to simulate a “SZMD” that reveals only *blocks of two significant digits* of a target number, without ever exposing the complete value of that number. The idea of the program is described in pseudocode terms in Algorithm 1. This program is **merely a simulation** of a machine to be built, using large hardware.

X.3 General idea of the program **szm_device.py**

The program implements a **proof of concept** of the SZM. The central premise is the **unreachability of the number under study**: the target value — a high-precision decimal number — is *never* stored as an attribute of the object. It is immediately decomposed into three decoupled components:

1. normalized mantissa $m \in [1, 10)$,

2. scientific exponent e ,
 3. sign,
- and the original reference is explicitly destroyed (`del _value`).

From this decomposition, the object behaves as a **sensor**: at each interaction it “reads” **two significant digits** of the mantissa and applies a *zoom + translation* operation over the residue, bringing the next digits to the front. The process repeats until the residue falls below a minimum detectable threshold, defined by the requested precision.

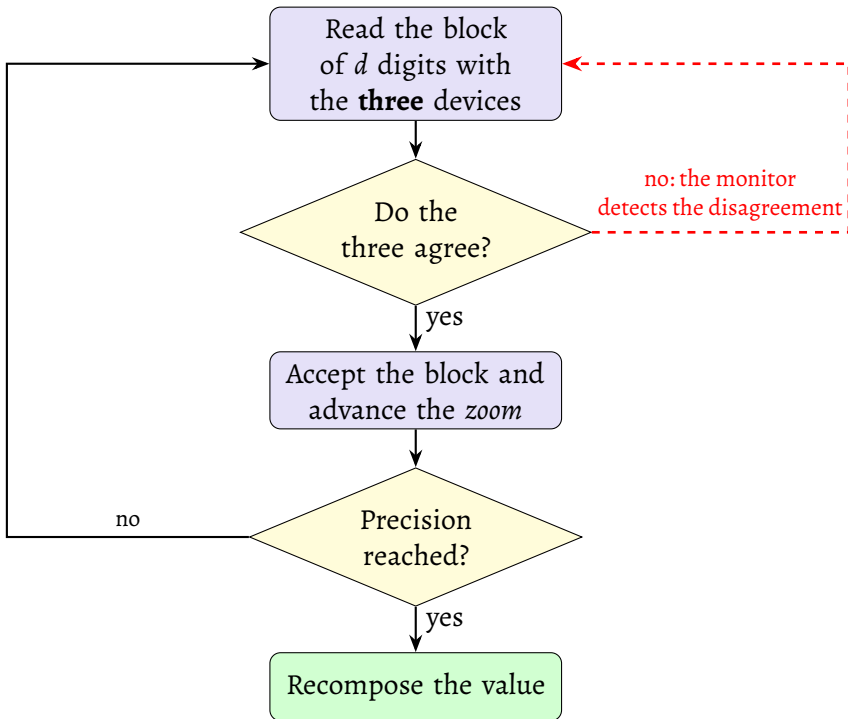


Figure 7 – Protocol of the simulated SZM. The three devices read the same block of digits; as long as there is no unanimity, the monitor orders a reprocess; once there is, the block is accepted and the *zoom* advances. The loop ends only when the requested precision is reached — this is why the value is always recovered, at the cost of time alone.



X.3.1 Flow of each step

Given the current mantissa m , each iteration performs:

1. **Extraction** of the first two significant digits: $d_2 = \lfloor 10 m \rfloor$.

2. **Computation** of the value to subtract: $next = d_2/10$.
3. **Zoom + translation**: $m \leftarrow 100(m - next)$, which shifts the next two digits into the reading position. *Note*: after this step, the residue m belongs to the interval $[0, 10)$, and may be smaller than 1 when the second digit of the extracted block was zero (for example, if $d_2 = 10$, the residue starts with 0, . . .). The invariant $m \in [1, 10)$ holds only for the first iteration; in the others, the relevant condition is $m \geq 0$.
4. **Numerical cleanup**: if $|m| < threshold$, then $m \leftarrow 0$ and the process ends, avoiding the leakage of spurious digits due to finite-arithmetic noise.

X.3.2 Structure of the class SZMDevice

The class defines:

- **Precision** (`_precision`): adjusts `decimal.getcontext().prec` and determines the minimum detectable threshold (`_threshold = 10^{-(precision-1)}`).
- **Minimal internal state**: only `_mantissa_norm`, `_exponent`, and `_sign`. There is no attribute that contains the complete number.
- **Restricted public API**:
 - `precision` — read-only property;
 - `extract_pairs()` — incremental reading; returns a dictionary with the blocks of two digits (`blocks`), the residues after each step (`mantissas`), and the cumulative concatenation of the blocks (`cumulative_digits`);
 - `recompose(block_list)` — reconstructs an approximation of the number *exclusively* from the blocks already revealed.

X.3.3 Shielding mechanisms

To ensure unreachability, the program employs five complementary mechanisms:

1. **Reference destruction**: the local variable that receives the target number is removed with `del _value` right after normalization.
2. **Minimal, decoupled state**: only the three components (mantissa, exponent, sign) are kept; none of them, in isolation, allows reconstructing the original value.
3. **API without full exposure**: there is no public method that returns the complete number. The `recompose` function depends only on the blocks already

extracted by the device itself.

4. **Cleanup threshold:** the `_threshold` parameter ends the extraction before residual digits can leak through numerical imprecision.
5. **`__slots__`:** restricts the instance's attributes, preventing the accidental injection of new fields that could circumvent the no-storage policy.

X.3.4 Demonstration

The main block (`if __name__ == "__main__":`) instantiates the device with `precision = 20`, runs `extract_pairs()`, and prints a table with the three returned lists. It then calls `recompose` to display the final result using only the blocks read. The output illustrates that the external application obtains progressive approximations of the number, but *never* the complete internal value — the characteristic that, in the context of the SZM, we call **unreachability**.

X.4 Feasibility simulation `szm_feasibility.py`

Although **`szm_device.py`** is sufficient to expose the principle of the SZM — the incremental extraction of the mantissa by pairs of digits — the practical question remains: *do the noise and imprecision of analog devices compromise the method?* To investigate this hypothesis in a controlled way, the program in Listing AG.8, **`szm_feasibility.py`**, was developed, incorporating a probabilistic error model and a decision protocol with redundancy. This program emulates deliberately imprecise hardware, so that failures are revealed and analyzed systematically.

The decision protocol is simple and is summarized in the flowchart of Figure 7: each block of digits is read by **three** independent noisy devices; as long as the three do not agree, a *monitor* detects the disagreement and orders a *reprocess* (reread); once there is unanimity, the block is accepted and the *zoom* advances. Since the loop releases each block only when there is agreement, the value is always recovered — the price of a bad device is only the *time* spent on reprocessings, never the loss of the result.

X.4.1 Results and interpretation

Table 2 summarizes the method's performance according to device quality. Three readings of the same block are made by noisy devices; as long as the three do not

agree, the monitor orders a reread — so the value is always recovered, at the cost only of *time* (reprocessings).

Table 2 – Feasibility of the SZM by device quality (prec. 20, 20,000 sessions per scenario, three devices with a unanimity protocol that *reprocesses until success*). The value is always recovered, except for the extremely rare *unanimous error* (the three devices agree on the *same* incorrect value): hence the recovery rate of 99.99 % rather than 100 %. The recommended operating point (green) is a cheap 3½-digit device: it recovers with negligible reprocessing and in fewer rounds than a 2-digit one. Going up to 4 digits saves only two rounds, without justifying the cost.

Scenario	d (dig./read)	noise on last digit	rounds	recovery rate	reproc. per block
Terrible	2	10 %	10	99.985 %	0.42
Poor	2	5 %	10	99.995 %	0.19
Reasonable (2 digits)	2	2 %	10	99.995 %	0.06
Cheap: 3½ digits	3	2 %	7	99.995 %	0.06
4 digits	4	2 %	5	99.995 %	0.06
3½ digits, less noise	3	1 %	7	100.000 %	0.03

In broad terms:

- **Nearly guaranteed recovery.** Across the whole range — from the terrible device to the cheap one — the value is recovered in about 99.99 % of the sessions. What escapes is the *unanimous error*, an event in which the three devices err toward the *same* value; it is extremely rare and, by definition, invisible to a majority monitor.
- **The cost is time, not error.** The worse the device, the more reprocessings per block — from ~0.03 (good) to ~0.4 (terrible) — and the higher the worst-case latency. The method does not stop working; it becomes slower.
- **There is a cheap operating point.** A 3½-digit device, with noise restricted to the last digit, recovers with negligible reprocessing and in fewer rounds than a 2-digit one. Going up to 4 digits saves few rounds and does not justify the hardware cost — it is the knee of the cost/performance curve.

In summary, the simulation demonstrates the **practical feasibility** of the SZM: even with cheap and noisy analog components, the minimal redundancy of three devices and reprocessing on disagreement suffice to recover the value with arbitrary

precision.

X.5 Speed and precision of the SZM vs. the precision of the digital computer

The SZM, by construction, has arbitrary precision. Therefore, if we compare the SZM with any Digital Computer, the SZM wins on precision, since it can work with any number of digits.

There is, however, an important issue here, namely the assembly of the digits to compose the numbers (multiple precision), which has a cost in speed. The native precision of a digital computer is fixed at the manufacture of its components. For example, 64 binary digits. In this case, the digital computer processes a numerical operation in just one or a few machine cycles⁷.

In the case of the SZM, in the assembly of the digits, it is necessary to combine the digits: if the number has 10 decimal digits and the SZM can obtain only a single digit at a time, 10 operations will be needed, with each operation possibly corresponding to several machine cycles. But this is an open question, since analog computers have very fast analog circuits⁸ and they perform the computation directly, whereas digital computers need to solve the differential equations using numerical computation algorithms. In numerical methods for solving ordinary differential equations, one of the fundamental parameters to be considered is the number of function evaluations per step, that is, how many times the function $f(t, y)$ — which defines the dynamics of the system — needs to be calculated at each iteration. This number directly influences the computational cost of the method, since each evaluation can be simple or extremely expensive, depending on the complexity of the problem. For example, the Euler method requires only one evaluation per step, whereas the classical fourth-order Runge–Kutta method requires four, which makes it more costly in terms of computation, but also more precise. Thus, the choice of the appropriate method depends on a balance between the cost of each step and the desired precision, this being a central parameter for the analysis of numerical efficiency. In a comparison between the numerical method, on a digital computer,

⁷ The number of cycles depends on the operation to be performed (addition, division, etc.) and on the hardware, but can vary between 1 and 35 cycles. In this analysis, we are not considering parallelism (GPUs, coprocessors, etc.).

⁸ The main components of an analog computer (GPAC) are the operational amplifiers, usually abbreviated as OPAMP (*Operational Amplifier*).

and an analog method, on an analog computer (GPAC — *General-Purpose Analog Computer*), we need to limit our comparisons to the solution of differential equations, and at this point the digital computer will have to use thousands of machine cycles to solve the differential equation⁹, whereas in the SZM, the only demand will be to join the computed digits.

One might think of using multiple-precision numbers on the digital computer¹⁰. What these libraries do internally is treat the numbers as if they were strings (or contiguous spaces allocated in memory), and when the mathematical operations are performed, instead of the processor carrying out the computation with a number of 64 native binary digits, it calls the mathematical algorithms to perform the operations digit by digit. But the great problem of this approach is really performance. The number of function evaluations per step is very large. In the SZM, digit-by-digit handling is done only for the assembly of the final result, whereas in a numerical method that used one of these multiple-precision libraries, all the mathematical computations would go through these digit-by-digit operations. These libraries find application in situations where performance is not an issue. Here itself, the **decimal** library was used in the program `szm_device.py` (Listing AG.7, discussed in Section X.2.4), for a proof of concept of the SZM.

Considering only the numerical solution of differential equations, the numerical precision of the digital computer is not the precision of the machine, for example 64 binary digits. The numerical precision is given by the order of the global error, as shown in Table 26 (Vol. II), Page 427. There is no point in working with 10 decimal digits if the numerical method is based on a Taylor series truncated at the fifth decimal place. If the maximum precision of the digital computer is 64 bits in floating point, which corresponds to about 16 significant decimal digits, in practice the precision of the digital computer in solving problems using numerical methods is far lower than those 16 digits. For a detailed discussion of stiff problems and the choice of appropriate numerical methods, see Appendices V (Vol. II) and W (p. 407).

Another detail is that the SZM can be applied to an entire “analog run,” whereas numerical methods need to be computed step by step of integration.

⁹ Depending on the differential equation to be computed, the number of cycles can be immense. Stiff problems, for example, require very robust but very computationally costly numerical methods. In some cases, the program needs to solve a system of algebraic equations at each step of numerical integration. In some cases, with variable step size, the step size can be reduced to such a small value that it reaches the precision of the machine (**UNDERFLOW**).

¹⁰ For example, some Python libraries for computations with multiple-precision numbers are `decimal`, `mpmath`, and `gmpy2`, and some prominent C++ libraries for multiple-precision computations are `GMP`, `MPFR`, and `Boost.Multiprecision`.

Table 3 shows a comparison between the speed-champion CPUs and some operational amplifiers (OPAMPs), which are the main devices in analog computation. The OPAMP LM741, from the 1970s, is the canonical example of the integrated circuit that the operational amplifier reached after successive generations — vacuum tubes in the earliest models, then discrete-component assemblies (George A. Philbrick Researches, 1966). But its performance was relatively low, with a rise time of 350 ns. The modern, ultra-high-performance OPAMPs, on the other hand, achieve rise times on the order of 0.020 ns (20 ps) — but that is the signal edge, and not a complete computation: Box 6 shows, on four independent grounds, why those amplifiers do not compute.

As for the parallelism that we see in digital computation, it can also exist in the SZM.

Table 3 – Comparison between the speed of digital processors and conventional and ultrafast Operational Amplifiers (analog OPAMPs).

Digital Speed Champions			
Processor / System	Max. clock	Duration of 1 cycle	References
AMD FX-8370 (record overclock)	8.723 GHz	0.115 ns	*
Intel Core i9-14900KS	6.2 GHz	0.161 ns	\$
AMD Ryzen 9 7950X3D	5.7 GHz	0.175 ns	\$
JUPITER (ARM Neoverse V2)	3.0 GHz	0.333 ns	†
Aurora (Intel Xeon Max)	2.4 GHz	0.417 ns	†
El Capitan (AMD EPYC Genoa)	1.8 GHz	0.556 ns	†
Conventional OPAMPs and Analog Speed Champions			
Operational Amplifier	GBW / typical bandwidth	Rise time	References
LM741	1 MHz	350 ns	†◇
TLO81	3 MHz	117 ns	†◇
OPA690	535 MHz	0.65 ns	†◇
AD8000	650 MHz	0.54 ns	†
OPA855	8 GHz	0.17 ns	* ◇
RF OPAMPs/LNAs ^a	20 GHz	0.020 ns	‡

Legend: * datasheet value; † $t_r \approx 0.35/f_{3dB}$; ‡ limited by slew rate; \$ nominal turbo/maximum clock; ◇ small signal.

Note on the axis of comparison: for the CPUs, the time column gives the duration of one cycle — the interval at the end of which the result is complete in the register. For the amplifiers, it gives the rise time, which measures when the edge *passes* 90% of the final value, and not when the output is that value within the tolerance with which it will be read. For an analog computing block, whose output is read at each *zoom*, the figure of merit is the settling time to a given precision, appreciably longer than the rise time. The two columns are therefore not interchangeable.

^a **Examples in the 20 GHz range:** ADI HMC519LC4 (18–31 GHz); Qorvo TGA2526 (2–20 GHz); Qorvo TGA2567-SM (2–20 GHz); Qorvo QPA2735 (13.75–20 GHz); Mini-Circuits AVA-0233LN+ (2–30 GHz).

Note: With RF LNAs/amplifiers in that range, the “analog computation” is performed as networks in $50\ \Omega$ (summation/combination, gains/attenuations, mixing, filters) and synthesis of in-band LTI ODEs — not as an ideal closed-loop opamp. Integration/differentiation are in-band approximations, without DC, requiring care with stability, gain/phase flatness, and linearity.

6. Why the fastest amplifiers in the table cannot compute

The last row of Table 3 records RF amplifiers below 20 ps. It would be tempting to take them as the ceiling of analog computation — and that is a mistake. They are fast precisely because they sacrifice what analog mathematics requires. Four reasons, all independent:

1. **There is no gain reserve.** A precise analog operation is imposed *from outside*, by the feedback network, and comes out exact only because the internal open-loop gain is colossally larger than the desired closed-loop gain. RF amplifiers have low, fixed internal gain: without that surplus, there is no way to force them to obey the equation.
2. **Impedance destroys the computation.** An ideal summer or integrator reads the voltage without drawing current. Above a few gigahertz, however, the whole board must respect the 50 Ω match, on pain of signal reflection — and matching impedance means *drawing* current from the input, which is exactly what the feedback network cannot tolerate.
3. **They do not see DC levels.** Integration requires reading and holding a steady voltage. Most ultrafast amplifiers are AC-coupled: if the signal stops oscillating, the output falls to zero.
4. **The feedback arrives late.** The signal must leave, travel along the trace, and return to the input in order to correct the amplifier. On a board, electricity advances about 15 cm per nanosecond; with rise times of 20 ps, the round trip through a few millimeters of copper is already too long. The device corrects a state that no longer exists and, instead of computing, oscillates.

The real ceiling is a decade earlier. Classical-architecture amplifiers in the tens-of-picoseconds range — the OPA855 (8 GHz gain bandwidth product) and the LMH5401 (8 GHz, fully differential and DC-coupled) — have a feedback network and do read DC levels: in principle, everything analog mathematics asks for. In practice, even there the design is hard. Two millimeters of trace behave as an antenna, a solder joint as an inductor, and the gap between traces as a capacitor, and these ghost components enter the equation uninvited: a multiplier by two becomes a filter. Moreover, the fastest ones are *decompensated* — the OPA855 is stable only at a gain of 7 or greater, which is stated in the very title of its data sheet —, so one cannot build a follower or a unity gain with them.

Where this arithmetic is in fact used. LiDAR sensors, calorimetry in particle physics, and the input stage of high-end oscilloscopes — the manufacturer itself lists optical time-of-flight and LiDAR among the OPA855's applications. In those cases the board

continued →

6. Why the fastest amplifiers in the table cannot compute (*continued*)

is designed with microwave engineering, and even the angle of a bend in a trace enters the calculation. Above that range, the classical analog computer is abandoned and the signal is manipulated as a radio wave: combiners to add, mixers to multiply — fast, but without the precision of a calculator.

Box 7 puts a hardware proof of concept to the reader: to build a low-cost circuit that extracts the digits of $\sqrt{2}$ by analog measurement rather than by conventional numerical computation.

7. A Challenge to the Reader: a low-cost physical proof of concept

What this is about.

The reader is invited to build a low-cost electronic circuit that solves the equation $x^2 - 2 = 0$, extracting the digits of $\sqrt{2}$ by *analog measurement*, and not by conventional numerical computation. The target is one thousand correct digits, or more.

The goal is not to compete with multiple-precision libraries, which perform this task better and faster. The goal is to demonstrate, in physical hardware, that the Successive Zooms Method allows an analog device with only three digits of precision to contribute to a result of arbitrary precision.

The target was chosen for the opposite reason to the one that might be supposed: being trivial and exactly known, it gauges the device without the problem interfering with the result. A proof of concept requires a target whose true value is reachable by an independent route — otherwise there would be nothing against which to compare the readings. The applications that motivate the method are the stubborn problems of Appendix V (Vol. II, p. 387), where the obstacle is stiffness, not the number of digits.

The idea in one sentence.

The analog device never measures a small number. At each stage the problem is *repositioned and rescaled* so that the quantity to be measured returns to the order of a few volts. The participant always measures something comfortable — for example, 3.4X V, discarding the non-significant digit X.

An image helps, taking the equator for what it is — a geometric circle, without line thickness, and not the relief of the planet. Suppose the problem were to determine the curvature of that circle. Over a football field, the arc is indistinguishable from a straight line. Subtract, however, the chord joining the two ends of the field, fixing

continued →

7. A Challenge to the Reader: a low-cost physical proof of concept *(continued)*

zero altitude at the edges, and magnify what remains vertically by a factor of 10^9 : one obtains an arc perfectly measurable with an ordinary tape measure. The instrument was not changed; the problem was.

And the procedure repeats upon itself. Once the arc is known to a few more digits, the circle is known to a few more digits; the new model, finer than the previous one, is then subtracted, and what remains is magnified once again. At every stage the figure is rigorously the same — a comfortable arc over a football field — and what changes is only the scale printed on the axes. That is why the tape measure never has to be replaced by a more precise one, at any depth.

The Successive Zooms Method proceeds exactly like this, digit by digit, indefinitely.

Targets.

Category	Correct digits
Bronze	≥ 6
Silver	≥ 100
Gold	≥ 1000
Honorable mention	above 1000, or a rigorous characterization of failure

Why the target is set so high.

The target of one thousand digits is not a performance requirement but a criterion of understanding. An implementation that correctly interprets the nature of the rescaling reaches arbitrary orders of magnitude at an approximately constant analog cost per block; an implementation that amplifies the residual instead of repositioning the problem stalls in the first few stages, through saturation or loss of resolution. The number of digits reached is therefore an indicator of the conceptual correctness of the setup, not of its constructional quality.

Put differently: whoever reaches twenty digits reaches one thousand by letting the circuit run for a few tens of milliseconds more, plus the cost of the integer bookkeeping, which grows with the number of digits already accumulated. Whoever does not reach twenty has an error of conception, not of components.

continued →

7. A Challenge to the Reader: a low-cost physical proof of concept *(continued)*

Assembly requirements.

1. **A control microcontroller**, with exact integer arithmetic for the accumulated coefficients. Fixed-precision floating point is not admissible in that bookkeeping.
2. **A digital-to-analog converter** and an **analog-to-digital converter**, of resolution not below 16 bits.
3. **An analog computing block**, able to form the residual of the current stage. Note that the analog operation required may vary with the depth of the stage; the participant is free in the choice of topology.
4. **Three independent monitors** reading the residual, each with a total reading error not exceeding 0.1% of full scale, over the range actually used, evidenced by the manufacturer's data sheet or by calibration against a traceable standard.
5. **Effective independence of the monitors**, ensured by individual electromagnetic shielding, galvanic ground isolation, supply from separate sources — preferably batteries, which are low in noise — and independent voltage references.
6. **Persistent logging** of the data to a file, written to a memory card attached to the microcontroller or to a host computer.

Why three independent monitors.

Validation of each block is by unanimity of the three, which presupposes uncorrelated errors. A shared voltage reference, common thermal drift or common-mode interference produce the *unanimous error* — three identical readings, all of them wrong — which a majority of three cannot detect. The empirical estimate of the correlation coefficient ρ between the monitors is, in itself, a result of interest. The same assumption reappears later with a different consequence: in the monitors, a shared source defeats *detection*; in the cascade of virtualized modules discussed below, it defeats *dilution* — which is why the error-accumulation model presented there holds in the uncorrelated case. Note further that unanimity validates the *reading*: the chain that deflates and accumulates the digits is single and comes after the vote, beyond its reach. No test prescribed here exercises it against a known value — the mandatory control test described below demonstrates the necessity of the analog part, not the correctness of the digital one —, but the per-stage log described below contains the three readings and the measured quantity, and allows the chain to be recomputed *off-line*.

continued →

7. A Challenge to the Reader: a low-cost physical proof of concept *(continued)*

Data to be logged.

Per stage: the block of digits obtained; the reading of each of the three monitors; the physical quantity actually measured and its order of magnitude; the discrepancies detected; the number of attempts; the ambient temperature; the measured reference voltage. At the end of the session: the complete chain of accumulated digits, in a single file.

Logging the quantity measured at each stage is not a formality. It is the datum that allows the analog contribution to be distinguished from the digital bookkeeping, a distinction on which the whole evidential value of the submission rests.

Mandatory control run.

Replace the analog reading with a random value, leaving the rest of the *firmware* untouched. If the digit chain survives, the analog block was decorative and the submission does not demonstrate the method. If it breaks, the necessity of the analog reading is demonstrated by its absence. It costs one line of *firmware* and constitutes the cleanest evidence available against the claim of disguised multiple precision.

Shielding the target.

The value of $\sqrt{2}$ must not appear at any point of the control loop, not even in debug assertions or in initial search values.

This is a precaution against involuntary leakage of information, not a requirement of a fiduciary nature. A *firmware* that consults the answer during acquisition turns the demonstration into a tautology, however entirely legitimate the participant's intent; the typical case is a `sqrt(2.0)` forgotten in a debug line.

Logging the digits to a file and subsequently checking them in Python is the intended and desirable procedure. There is no restriction whatsoever on how the data travel, whether by serial port, memory card or any other means.

Verification of the result.

Verification is performed **exclusively offline**, after the acquisition session has ended. The logged digits are checked by a Python program that compares them against the reference value, obtained by exact integer square root — with no floating point and no rounding. The verifier uses only the standard library, so nothing needs to be installed, and it is reproduced in Listing AG.11, whose QR code leads to the executable file.

continued →

7. A Challenge to the Reader: a low-cost physical proof of concept *(continued)*

Part two: characterizing the breakdown.

Submissions to Part II are accepted on their own, with or without a submission to Part I. The participant is asked to determine experimentally: the per-block error rate, ε_k , as a function of the stage index k ; the stage at which the setup ceases to recover correct digits, and the physical cause identified; the empirical correlation coefficient ρ between the three monitors; and the sensitivity of the result to temperature and to drift of the voltage reference.

Refutation criteria, declared *a priori*.

The central thesis of the method is considered refuted if the following is found to hold reproducibly: monotonic growth of ε_k with k , indicating a non-contractive recursion; a need for increasing digital-to-analog converter resolution with k ; collapse of the recovery rate for moderate values of ρ ; or consistently obtaining fewer than one correct digit per stage. Note, as to the last, that a **multiple root** — one at which the function and its derivative vanish at the same point — produces that symptom without the method being at fault: for multiplicity m , the residue is proportional to the m -th power of the error, so that the extraction rate falls by a factor of m (with $m = 2$, each pair of digits of the residue yields a single digit of the root), and for even m the residue does not even change sign. The degradation is there a property of the problem *as presented to the method* — extraction over f/f' , or with m known, recovers the rate —, and not of the recursion. The criterion refutes when the symptom appears at a simple root.

Negative results are equally publishable. A submission that rigorously documents failure at a given stage, identifying the physical cause, is worth more than one claiming a high digit count without thermal records and without logs from the three monitors.

What is to be submitted.

The *firmware* actually executed, in a public repository and with its version identified; a legible electrical schematic and a photograph of the build; a component list with manufacturer, model and batch; the calibration procedure actually used; the ambient conditions of the session; the raw *log* file; an account of failures and intermediate adjustments. This last item is asked for explicitly, since calibrating the analog settling time is the most frequent failure mode.

continued →

7. A Challenge to the Reader: a low-cost physical proof of concept (continued)

On the public repository.

Publishing the material in a public repository under version control is a requirement, not a convenience. It is what the results page will link to; it is what allows third parties to rebuild the setup; and it is the version history that distinguishes what was actually executed from what was adjusted afterwards. Maintaining the repository is the participant's responsibility.

Reproducibility.

A submission must contain enough detail for an independent third party to rebuild the setup and obtain equivalent results without having to consult the participant. It is worth being precise about what equivalent means, since not everything should coincide: the digit chain must be *identical*, being mathematical and independent of the setup; the measured voltages, the breaking stage and the per-block error rate, however, depend on components, on calibration and on ambient conditions, and of these only agreement within the declared tolerances is expected. A submission that does not permit independent reconstruction will be recorded as a report, not as a result.

Publication and rights.

The best results will be published at <https://www.impnet.com.br/bolholandia/mzs-resultados/en/>>, with a consolidated table of digits reached, breaking stage and estimated ρ . The material of each project remains hosted in the participant's own public repository; the page publishes only the consolidated index, linking to each of them. The preferred language for submissions is English, so that the result reaches the largest number of readers; submissions in Portuguese are equally accepted. Copyright remains entirely with the authors of the respective projects, who are free to publish their results independently.



X.5.1 Recurrent errors of interpretation

The misconceptions gathered in Table 4 were all observed in real discussion of the method. They do not arise from carelessness: they arise from intuitions that are cor-

rect in other contexts, imported into a domain where they do not hold. Participants are advised to go through the list before starting the build.

Table 4 – Recurrent errors in interpreting the SZM.

Error	Correction
<i>Taylor bias.</i> Supposing that magnifying the neighborhood of a simple root can only reveal linearity.	True for isotropic, passive magnification. The SZM subtracts the known affine part <i>before</i> rescaling; no slope remains to be amplified, since it was removed, not scaled.
<i>Imported physical floor.</i> Concluding that the recursion will run into noise, granularity or finite resolution.	That floor belongs to the physical object, not to the method. The target of the SZM is an exact equation; each stage produces another exact equation. The analog device is merely the instrument for a local three-digit reading, in the case of an instrument capable of reading three digits. ¹¹
<i>Confusing a mathematical target with a physical quantity.</i> Applying the zooms to a mass, a length or a voltage.	The zooms act on the equation. Physical analogies serve to disarm the mistaken intuition that there is nothing left to measure; they do not serve to establish the limits of the method.
<i>Requiring a converter of increasing resolution.</i> Presuming that stage k demands more bits than stage 1.	The quantity measured is always of the order of a few volts. Resolution that grows with k indicates an implementation that amplifies the residual instead of repositioning the problem, and is among the refutation criteria.

¹¹ More than three digits would increase the performance of the SZM, since the coupling would proceed two digits at a time (or three at a time) and the monitors would detect fewer errors, but this represents a greater financial cost.

Error	Correction
<i>Measuring the raw residual.</i> Trying to read directly a quantity of order 10^{-k} .	A small number is never measured. After deflation and rescaling, one reads 3.4X V, discarding the non-significant digit X.
<i>Floating point in the bookkeeping.</i> Accumulating coefficients in double.	Deflation is the cancellation of nearly equal quantities and requires exact integer arithmetic, where there is no digit to lose. In fixed-precision floating point the method fails by construction.
<i>Cancellation in the analog domain.</i> Trying to form the difference with an analog summer.	No summer cancels 10^{200} against 10^{200} while preserving the difference. Deflation is necessarily digital; the division of labour is imposed by physics, not chosen for convenience.
<i>Leaking the target.</i> Leaving <code>sqrt(2.0)</code> in a debug assertion or as an initial search value.	It turns the demonstration into a tautology, however honest the intent. See the shielding of the target, in Box 7.
<i>Nominally independent monitors.</i> Sharing a voltage reference, ground or thermal enclosure.	It produces the <i>unanimous error</i> : three identical readings, all of them wrong, undetectable by any vote among the three.
<i>Expecting a complexity advantage.</i> Supposing that the SZM beats the schoolbook square-root extraction in asymptotic cost.	It does not: the integer bookkeeping is $O(k)$ per stage, just as that method is. The thesis concerns the <i>nature</i> of the contribution, not speed.

Error	Correction
<i>Treating the thousand-digit target as performance.</i> Optimizing for speed in order to reach it.	It is a criterion of understanding. Whoever reaches twenty digits reaches one thousand by letting the circuit run somewhat longer, without altering the build.

X.6 Richardson extrapolation/analysis applied to the SZM

Richardson Extrapolation (ZLATEV et al., 2017) is a numerical technique that consists in combining results obtained with different step sizes to eliminate dominant error terms, thereby increasing the order of precision of a computation. This procedure can be used in two main ways:

- 1. **Improving precision:** refining the step and combining results appropriately, eliminating the dominant error term.
- 2. **Estimating accuracy and the order of convergence:** obtaining the effective order p of a numerical method and estimating the limit value that would be reached as the step tends to zero.

This Richardson extrapolation can be carried out entirely independently and in parallel with the SZM, functioning as a monitoring system: it tracks the results and only intervenes if it detects relevant discrepancies. Since the data from different refinements are already available, the application of the method is extremely fast, as it involves only simple arithmetic operations.

X.6.1 Step, zoom levels, and analog execution (“run”)

The relation

$$h \propto 2^{-Z}$$

(X.6)

expresses that the step size h decreases exponentially with the zoom level Z . Introducing the proportionality constant C , we have¹²

$$h = C 2^{-Z}. \quad (\text{X.7})$$

Each unit increment in Z reduces h by half, increasing the resolution of the analysis.

In the context of the SZM, we distinguish two operational regimes:

Static processing.

The level Z is fixed before execution, and the step h remains constant during the computation. The results of multiple levels ($h, h/2, h/4, \dots$) can be obtained as independent “snapshots” and then combined by Richardson extrapolation (see Equation (X.10)) to estimate the limit value and the effective order.

Dynamic execution (“analog run”).

As shown in Figure 8, in a classical analog computer (GPAC — *General-Purpose Analog Computer*), as described, for example, by Jackson (1960, p. 58–59), the simulation evolves continuously in physical time:

Classical cycle: → **STANDBY** → **IC/RESET** → **COMPUTE/OPERATE** → **HOLD** → **REPETITIVE SOLUTION** → (returns to **STANDBY** ...). Each of these execution blocks can be designated as *states* of the analog computer (GPAC).

Beyond the classical analog cycle, the diagram in Figure 8 also shows what the parallel monitoring by the Richardson method would look like, in an “analog run.”

A modern hardware implementation of the SZM can reuse the concept of some of these states and, at the same time, take advantage of digital computation and other recent technologies. Today, the wide availability of high-performance, low-cost electronic components allows modern computers, integrated with microcontrollers or FPGAs, to automatically control all these states, while dedicated hardware continuously monitors the **COMPUTE/OPERATE** state.

For this, measurements can be made by minimally intrusive *sample-and-hold* at instants t of interest (e.g., $t = 15$ s), with finer levels (greater Z) corresponding to denser temporal/spatial samplings (smaller h). In this configuration, Richardson

¹² The factor 2^{-Z} is a typical convention in hierarchical refinement schemes (binary mesh, multi-resolution levels, comparison $h, h/2, h/4$ in Richardson extrapolation (RICHARDSON, 1911; ZLATEV et al., 2017)). In the SZM, other reduction factors (for example, 3^{-Z}) can be adopted, provided the mapping between Z and h is known and stable.

extrapolation acts as an external and *parallel* monitor: it computes p and the extrapolated value from the samples of multiple levels and only “intervenes” when it identifies discrepancies above tolerances (following sections, Equations (X.9) and (X.11)). Once the data are available, post-processing is very fast, since it involves only simple arithmetic operations.

X.6.2 Error model and working hypothesis

Suppose the SZM produces an approximation $Q(h)$ for a scalar (or vector) quantity $Q^\star$, where $h > 0$ is an effective refinement parameter of the SZM (e.g., step, internal resolution, or $h \propto 2^{-Z}$, with Z the zoom level). We assume the asymptotic regime:

$$Q(h) = Q^\star + C h^p + C_2 h^{p+1} + O(h^{p+2}), \quad (\text{X.8})$$

with constants C, C_2 independent of h and order of convergence $p > 0$. For vector outputs, replace differences by norms, $\|\cdot\|$.

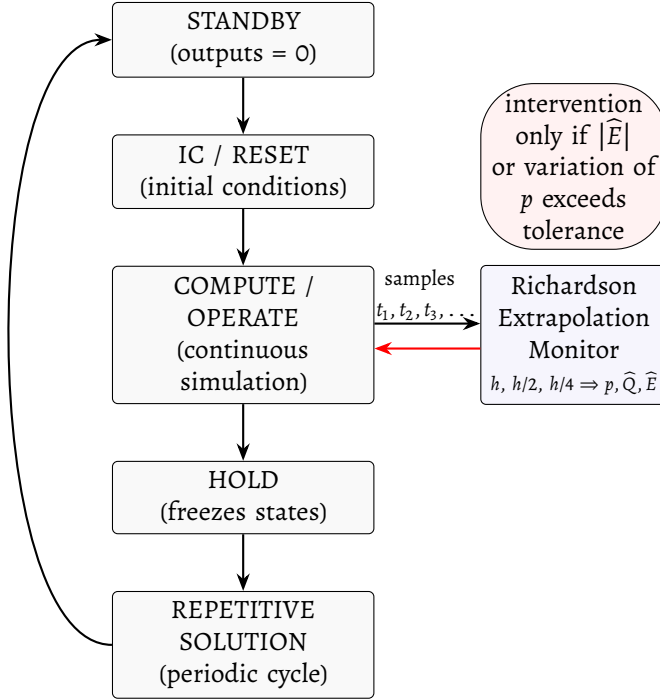


Figure 8 – Operation flow of the “analog run,” considering a classical analog computation model, adapted for parallel monitoring.



X.6.3 Estimation of the order p

Consider three levels h , $h/2$, $h/4$ and the corresponding approximations Q_h , $Q_{h/2}$, $Q_{h/4}$. In the regime of Equation (X.8),

$$p \approx \log_2 \left(\frac{\|Q_h - Q_{h/2}\|}{\|Q_{h/2} - Q_{h/4}\|} \right). \quad (\text{X.9})$$

When Q is scalar, the norm is the absolute value.

X.6.4 Extrapolation for $h \rightarrow 0$

With p known (or estimated by Equation (X.9)), the two-level Richardson extrapolation provides

$$\widehat{Q} \equiv \mathcal{R}(Q_h, Q_{h/2}; p) = \frac{2^p Q_{h/2} - Q_h}{2^p - 1}, \quad (\text{X.10})$$

eliminating the dominant term $C h^p$ of Equation (X.8). An estimate of the error in $Q_{h/2}$ is

$$\widehat{E}_{h/2} \approx \widehat{Q} - Q_{h/2} = \frac{Q_{h/2} - Q_h}{2^p - 1}. \quad (\text{X.11})$$

X.6.5 Practical protocol for the SZM

1. Define a sequence of refinements $h_k = h_0 2^{-k}$ (or levels $Z_k = Z_0 + k$ with $h_k \propto 2^{-Z_k}$), $k = 0, 1, 2, \dots$
2. Run the SZM and measure Q_{h_k} . For vector/field outputs, choose a norm coherent with the problem (e.g., spatial L^2).
3. For each consecutive triple (h_k, h_{k+1}, h_{k+2}) , estimate p_k via Equation (X.9).
4. With p_k , compute the extrapolation $\widehat{Q}_k = \mathcal{R}(Q_{h_k}, Q_{h_{k+1}}; p_k)$ by Equation (X.10) and the estimated error $\widehat{E}_{k+1}$ by Equation (X.11).
5. Stopping criteria: stop when $\|\widehat{Q}_k - \widehat{Q}_{k-1}\|$ and $|p_k - p_{k-1}|$ are smaller than predefined tolerances, or when $\|\widehat{E}_{k+1}\| \leq \varepsilon$.

X.6.6 Illustrative numerical example

Suppose the SZM, applied to a specific problem, produces:

$$\begin{aligned} Q_{h=0.1} &= 1.5342, \\ Q_{h=0.05} &= 1.5128, \\ Q_{h=0.025} &= 1.5060. \end{aligned}$$

The estimated order is:

$$p \approx \log_2 \left(\frac{|1.5342 - 1.5128|}{|1.5128 - 1.5060|} \right) \approx 1.65.$$

The Richardson extrapolation with $h = 0.1$ and $h/2 = 0.05$ provides:

$$\widehat{Q} = \frac{2^{1.65} \cdot 1.5128 - 1.5342}{2^{1.65} - 1} \approx 1.5028.$$

The estimated error in $Q_{h/2}$ is:

$$\widehat{E}_{h/2} \approx \frac{1.5128 - 1.5342}{2^{1.65} - 1} \approx -0.0100.$$

This procedure indicates that the limit value for $h \rightarrow 0$ is close to 1.5028 and that the solution at $h = 0.05$ has an error of the order of 1.0×10^{-2} .

X.7 Implicit methods, hybrid methods and the verification of the SZM results

A numerical method is said to be **implicit** when, to determine the value at the next integration step, it is necessary to solve an equation in which that value itself appears on both sides of the equality. This generates a circular dependence: the value one wishes to compute is already present in the definition of the expression used to compute it.

As an example, consider the **Implicit Euler Method**, applied to an ordinary differential equation of the form

$$\frac{dy}{dt} = f(t, y).$$

At the integration step, the update is given by:

$$y_{n+1} = y_n + h f(t_{n+1}, y_{n+1}),$$

where h is the time step.

Note that the function f depends on the unknown value y_{n+1} , which appears both on the left-hand side of the equation (as the unknown to be determined) and on the right-hand side (as an argument already needed for the computation). This is the characteristic trait of an implicit method.

To solve this equation at each step, an auxiliary function is defined:

$$g(y_{n+1}) = y_{n+1} - y_n - h f(t_{n+1}, y_{n+1}),$$

and its root is sought. The problem then becomes one of finding y_{n+1} such that $g(y_{n+1}) = 0$.

In practice, one typically uses the **Newton Method** (or variants) to determine this root. Each Newton iteration approximates the desired solution, requiring the computation of the derivative of g (or of the Jacobian, in the case of systems) and successive corrections until convergence. This need to solve a root-finding problem at each integration step is what makes implicit methods more costly than explicit methods, but also gives them their superior stability properties.

X.7.1 Hybrid analog-digital methods as support for implicit methods

One proposal to avoid having to use the Newton Method at each integration step is the use of **analog circuits** as a source of a first approximation for the implicit solution. In this scenario, the circuit acts as a kind of *first opinion*, quickly providing an initial value close to the sought root. The Newton Method would then be employed only to refine this result. If the precision of the analog computation were sufficiently high, the digital refinement could even be dispensed with.

This approach differs from three lines already known in the literature:

1. **Classical analog computation**, used between the 1950s and 1970s, which solved entire ODEs directly in hardware (for example, the analog computers described in the books of George A. Philbrick Researches (1966), Truitt and Rogers (1960), and Jackson (1960)).
2. **Hybrid analog-digital computation**, developed in the 1960s to 1980s, whose objective was to accelerate specific computations (for example, the analog computers described in the book of Bekey and Karplus (1968)).
3. The techniques of **digital preconditioning**, today widely used in implicit methods, but always based on extrapolations, approximations of the Jacobian, or purely numerical heuristics.

The use of analog computation as a preconditioning stage for implicit methods has a direct precedent in Huang et al. (2017), discussed in Section X.17: there the analog accelerator supplies the initial guess and the digital solver refines it. The strategy of this section can be understood as a form of *Analog-Assisted Newton*, or as a *Hybrid Analog Iteration*, in which physical processing precedes and assists the digital refinement — sparing, at each step, the expensive part of the implicit method: the solution of the system with the Jacobian. The book's original contribution is therefore not that idea, but the **role inversion** developed in Section X.7.2: there the

digital side stops solving and only verifies, by direct substitution of the SZM result into the differential equation, with no iteration at all.

Such an approach connects three domains that until now had remained separate: the stability of implicit methods, Newton preconditioning, and the use of analog circuits. This integration suggests a promising path for reducing computational costs in stiff problems, opening room for new hybrid architectures.

X.7.2 Residual verification of the results obtained by the SZM

This same infrastructure can be used to **verify** the results obtained by the SZM — but not by re-solving the equation. If the stiffness of the system is such that not even implicit methods solve it efficiently, none of them can serve as judge: the solution of a low-order method has fewer correct digits than the candidate to be judged. What one does is what one does with any proposed solution of an equation: **substitute it into the definition itself**. For an initial-value problem the definition has two equivalent forms. The differential one, $\dot{y} = f(t, y)$, would require differentiating the sampled candidate numerically, and differentiation amplifies the error of each sample. The integral one, $y(t_b) - y(t_a) = \int_{t_a}^{t_b} f(s, y(s)) ds$, only integrates, and integration attenuates. It is the latter that is used: in a window $[t_a, t_b]$, the digital side receives the samples of y that the measurement provides, evaluates f at them and integrates it along the window; the residual $R = \Delta y - \int_{t_a}^{t_b} f ds$, so computed, must vanish within the digits delivered by the zoom cascade — together with the continuity of $y(t_a)$ with the previous window. The comparison tests exactly what can go wrong in an analog computer (function generators, tolerances, drift), with an evaluation of f independent of them. The digital side acts only as a monitor, in parallel and on the result already obtained: there is no Newton iteration, because nothing is being solved, and the SZM is not touched unless an error is detected. How and at which instants to sample, and with which rule to integrate, are decisions of measurement and implementation, outside the scope of this book.

X.7.3 Why use the second strategy and not the first?

The use of the first strategy, of Section X.7.1, entitled *Hybrid analog-digital methods as support for implicit methods*, is infeasible because it would require the construction, tuning, and synchronization of an analog computer (GPAC) with the numerical method, which would represent a rather high cost.

In the reverse strategy, of Section X.7.2, called *Residual verification of the results obtained by the SZM*, the SZM is born already as a **hybrid computer** — that is, all the elements necessary for generating the solution and validating it would already be integrated. In this way, the complexity of maintaining two separate systems (one analog and one digital) is avoided and, at the same time, the possibility of residual verification whenever necessary is guaranteed. Table 5 summarizes this comparison.

Table 5 – Comparison between the two proposed strategies.

	Strategy 1: <i>Hybrid analog-digital methods as support for implicit methods</i>	Strategy 2: <i>Residual verification of the results obtained by the SZM</i>
Advantages	• Reduces the number of Newton Method iterations. • Possibility of accelerating the solution of stiff systems.	• The SZM is born already as a hybrid computer. • The digital side acts only as verifier, by residual; nothing is re-solved. • Avoids duplication of infrastructure. • Continuous and automated validation. • A majority of “green lights” is expected.
Limitations	• Would require the construction of — a dedicated analog computer (GPAC). • Need for tuning and synchronization between analog and digital systems. • High cost and additional technical complexity.	
Validation / reaction mechanism	Not applicable: the numerical method is the main solver; there is no structured external validation stage.	• If validation indicates inconsistency, a “red light” goes on for the SZM. • The computation is redone/adjusted within the SZM itself; the numerical method does not replace the SZM.

X.8 SZM and other numerical methods on digital computers

The SZM has analog computation at its core. This allows the computation not to suffer the known practical computability restrictions of digital computation. The so-called “stubborn problems,” discussed in Appendix V (Vol. II), cease to be an obstacle

in analog computation — and this includes problems far more complex and considered intractable by digital methods. This means that, if you have an absurdly complex system of nonlinear differential equations, with billions of coupled equations, provided you have sufficient hardware for it, that the system is physically stable, and that the model is faithful to the physical system, you will be able to obtain the results. Such a scenario recurs in countless phenomena related to life, from chemical kinetics (organic and inorganic), through biochemical processes and molecular biology mechanisms, to metabolic network dynamics, gene regulatory networks (GRNs), cell signaling networks, tissue- and organ-level interactions, biological neural networks, and, finally, the multiscale molecular modeling that integrates all these levels.

Basically, analog computation seeks to physically replicate the physical system¹³. If the analogy does not work, the physical system probably will not work either. For example, the instabilities of the physical system¹⁴ would be replicated in the analog system.

In practice, however, there are issues of scale — and one must say where they live, because trained intuition looks for them in the wrong place (Box 4). In addition, traditional analog computation suffers from numerical imprecision, a problem solved by the hybrid approach of the SZM. Other hybrid analog-digital approaches have already existed (BEKEY; KARPLUS, 1968), but not in the sense of combining digits — which is the fundamental strategy of the SZM. During the combination of the numerical digits, and also throughout the progress of the computation in the SZM, countless adjustments to the SZMDs will be needed for the numerical rescaling of the digits and the advance of the computation, until the integration domain is reached.

If the SZM approach were applied to computers of the 1960s, thousands of engineers would be needed to manually adjust each configuration of the SZMDs at each zoom applied at each step of the integrations. There would be countless potentiometers to be calibrated, countless cables to be connected and reconnected, and a high probability of human error. All this would imply absurd costs and execution times — making the process completely infeasible at that time, due to the absence

¹³ Usually, this replication is done with electronic circuits whose equations are identical or similar to those of the physical system. The advantage is that, most of the time, electronic circuits are cheaper and more controllable than the physical system.

¹⁴ The instabilities of a physical system are not the same as the instabilities of a numerical method used to solve the problem, which often reflect limitations or peculiarities of the mathematical formulation that describes/models the system. One of the ways to deal with unstable physical problems is to recondition the physical problem itself, that is, to modify the real system so as to reduce or eliminate the unstable behavior.

of automation, of advanced digital control, and, above all, of Artificial Intelligence, which, in the SZM, is essential to replace these thousands of engineers with much more agility and precision.

X.8.1 The role of AI in the SZM

One of the fundamental requirements of the SZM is the existence of an artificial intelligence capable of dynamically configuring the SZMDs, defining their internal parameters and interconnecting them as needs arise during execution. Part of this logic can be deterministic (mechanized) and part can be learned, for example, via neural networks. Thus, if a differential equation needs to be solved, the basic configuration of an SZMD is known — there will be at least one integrator circuit. However, other adjustments such as scalings, gain, and zoom levels must be decided “on-the-fly,” as a function of the behavior obtained during execution.

In the classic analog-computation book of Jackson (1960, p. 60), written in the 1960s, the author observes:

The computer does not think. The person performing the study is the one who must think — the machine will do only what it is instructed to do through the programming procedure. This may seem too obvious to mention, but the frequent failures, arising from meaningless data and from long and expensive computer studies, are a painful reminder to people in the field that the obvious is often forgotten.

Indeed, such original analog machines had no form of reasoning whatsoever. Modern digital computers do not “think” like humans either, but they can already learn patterns, interpret natural language, and make decisions based on models. Thus, automatically translating a differential equation into an interconnected network of SZMDs — with successive adjustments of scale and zoom — is not trivial, but can be enabled with modern AI, especially in an adaptive execution environment such as the SZM. In this sense, AI is expected to evolve to completely replace the human designer, combining learning techniques with algorithms capable of automating the construction of circuits (for example, the automatic generation of circuits in the SPICE standard), something that is already beginning to appear in current electronic CAD tools, even without coupled AI.

Recent examples of this progress include the use of LLMs for the automatic generation of circuits in SPICE (NAU; KRUMMENAUER; ZIMMERMANN, 2026;

ZHANG et al., 2025), the application of reinforcement learning to the design of analog circuits (SETTALURI et al., 2020), and commercial AI-driven synthesis systems for PCBs and embedded electronics (Diode Computers, 2025).

The greatest effort is concentrated in the construction of the initial circuit associated with the definition of the differential equation(s). Once this configuration is generated, the successive zooms of the SZM come to occur only through adjustments to the parameters of the analog circuit, without the need for structural reconstruction.

X.8.2 Reduction of a linear ODE of order n to a system of first-order equations

Every linear ordinary differential equation (ODE) of order n can be reduced to an equivalent system of n first-order equations. This transformation is essential because it facilitates both the mathematical analysis and the implementation in analog circuits (Section X.8.3).

Consider the non-autonomous linear ODE of order n :

$$b_n(t) \frac{d^n y}{dt^n} + b_{n-1}(t) \frac{d^{n-1} y}{dt^{n-1}} + \cdots + b_1(t) \frac{dy}{dt} + b_0(t) y = g(t), \quad (\text{X.12})$$

with initial conditions

$$y(t_0) = c_0, \quad y'(t_0) = c_1, \quad \dots, \quad y^{(n-1)}(t_0) = c_{n-1}, \quad (\text{X.13})$$

and assuming $b_n(t) \neq 0$.

Step 1.

We isolate the derivative of order n :

$$y^{(n)} = a_{n-1}(t) y^{(n-1)} + a_{n-2}(t) y^{(n-2)} + \cdots + a_1(t) y' + a_0(t) y + f(t), \quad (\text{X.14})$$

with $a_j(t) = -b_j(t)/b_n(t)$ and $f(t) = g(t)/b_n(t)$.

Step 2.

We define the state variables:

$$x_1 = y, \quad x_2 = y', \quad x_3 = y'', \quad \dots, \quad x_n = y^{(n-1)}. \quad (\text{X.15})$$

Step 3.

Their derivatives satisfy:

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = x_3, \quad \dots, \quad \dot{x}_{n-1} = x_n,$$

and, by Eq. (X.14),

$$\dot{x}_n = a_0(t)x_1 + a_1(t)x_2 + \dots + a_{n-1}(t)x_n + f(t). \quad (\text{X.16})$$

Step 4.

The equivalent first-order system is:

$$\dot{\mathbf{x}}(t) = A(t) \mathbf{x}(t) + \mathbf{f}(t), \quad \mathbf{x}(t) = \begin{pmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{pmatrix}, \quad (\text{X.17})$$

with

$$A(t) = \begin{pmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \\ a_0(t) & a_1(t) & a_2(t) & \dots & a_{n-1}(t) \end{pmatrix}, \quad \mathbf{f}(t) = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ f(t) \end{pmatrix}.$$

Therefore, any linear ODE of order n can be represented as a linear system of n first-order equations.

Example 1. For

$$y'' + 3y' + 2y = e^{-t},$$

defining $x_1 = y$, $x_2 = y'$, we obtain:

$$\begin{cases} \dot{x}_1 = x_2, \\ \dot{x}_2 = -2x_1 - 3x_2 + e^{-t}. \end{cases}$$

Example 2. For

$$y^{(3)} - 4y'' + 6y' - 4y = \sin t,$$

defining $x_1 = y$, $x_2 = y'$, $x_3 = y''$, it results:

$$\begin{cases} \dot{x}_1 = x_2, \\ \dot{x}_2 = x_3, \\ \dot{x}_3 = 4x_3 - 6x_2 + 4x_1 + \sin t. \end{cases}$$

X.8.3 Conversion of differential equations into analog circuits

Once the ODE has been reduced to a first-order system, it can be translated into an analog circuit. The usual method is based on inverting integrators with operational amplifiers, since direct differentiation in hardware is not very robust. The procedure follows:

1. **Rearrange the equation:** Isolate the highest-order derivative, so that all the others appear on the right-hand side.
2. **Apply the reduction:** Convert each ODE into an equivalent system of first-order equations (Section X.8.2).
3. **Implement the blocks:**
 - Adders to compose the right-hand side of the equations.
 - Inverting integrators to reconstruct the lower-order variables from $y^{(1)}$. Each integration inverts the sign.
 - Gain multipliers (potentiometers or digipots) to implement coefficients.
4. **Generate the forcing function:** It can be realized by DC voltage sources (step), sinusoidal/square generators, or FPGA-based arbitrary-function generators.¹⁵
5. **Define initial conditions:** Introduced as initial voltages on the integrators.
6. **Scale variables and time:** Adjust amplitudes and time scales to keep voltages within the practical range of the hardware used — typically a few volts.

Thus, the resulting circuit diagram allows the solution of the ODE to be observed as a voltage as a function of time.

¹⁵ In arbitrary-function generators, the typical output stages are: (i) high-bandwidth CFAs (e.g., THS3092, AD811, LMH6702); (ii) high-speed voltage-feedback *op-amps* (e.g., AD8056, AD8066); and (iii) low-power rail-to-rail CMOS *op-amps* (e.g., TLV3092).

X.8.4 The logical multiplication (virtualization) of the SZMDs

Consider an analog computer (GPAC — *General-Purpose Analog Computer*) capable of solving continuous dynamical systems modeled by differential equations of the form:

$$\begin{cases} \dot{\mathbf{x}}(t) = \mathbf{f}(t, \mathbf{x}(t)), \\ \mathbf{x}(t_0) = \mathbf{x}_0. \end{cases}$$

8. Lipschitz condition

A function $\mathbf{f} : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^m$ is said to be **Lipschitz continuous** on D if there exists a constant $L > 0$ such that:

$$\|\mathbf{f}(\mathbf{x}_1) - \mathbf{f}(\mathbf{x}_2)\| \leq L \|\mathbf{x}_1 - \mathbf{x}_2\|, \quad \forall \mathbf{x}_1, \mathbf{x}_2 \in D.$$

Example of a Lipschitz function:

$$f(x) = 2x.$$

In this case:

$$|f(x_1) - f(x_2)| = 2|x_1 - x_2| \Rightarrow L = 2.$$

Thus, the ODE $\frac{dx}{dt} = 2x$ has a unique solution:

$$x(t) = x_0 e^{2t}.$$

Example of a non-Lipschitz function:

$$f(x) = \sqrt{|x|}.$$

Near $x = 0$:

$$\frac{|f(x) - f(0)|}{|x - 0|} = \frac{1}{\sqrt{|x|}} \rightarrow +\infty,$$

so no finite L exists. In the ODE

$$\frac{dx}{dt} = \sqrt{|x|}, \quad x(0) = 0,$$

there is no uniqueness of solution: both $x(t) \equiv 0$ and solutions of the type

$$x(t) = \frac{1}{4}(t - c)_+^2, \quad c \geq 0,$$

are valid, where $(\cdot)_+ = \max(\cdot, 0)$ denotes the positive part.

Source: prepared by the author based on (BOYCE; DIPRIMA, 2009).

By the Existence and Uniqueness Theorem¹⁶, if the function $\mathbf{f}$ satisfies certain conditions (for example, being Lipschitz in $\mathbf{x}$, as shown in Box 8), then the solution $\mathbf{x}(t)$ is unique (BOYCE; DIPRIMA, 2009, Chapter 2, Theorem 2.8.1).

Thus, if the analog system is interrupted at the instant $t = t_0$, and the state vector $\mathbf{x}(t_0)$ is measured with sufficient precision (within the tolerance required by the SZM), we can resume it later with equivalent behavior, simply by physically reinitializing the SZMDs with the same state vector. In integrating analog circuits, for example, this means recharging the capacitor with the voltage it had at the moment of interruption, within the precision range guaranteed by the method.

In highly complex problems (molecular modeling, metabolic networks, GRNs, multicellular tissues, biological neural networks, etc.), thousands of variables may be coupled, requiring the availability of thousands of physical SZMDs. From this perspective arises what we metaphorically call *the miracle of the multiplication of the SZMDs*¹⁷: a few real analog modules are combined with a modern digital computer (equipped with gigantic memories, CPUs, and parallel GPGPUs), which acts as an *orchestrator* and *virtual replicator*.

This technique resembles time-division multiplexing (TDM), but with the difference that TDM multiplexes channels in communication systems, whereas in the technique presented here the SZMDs are multiplied. The cost of this multiplication with temporal multiplexing is a loss of performance which, in the strictly sequential (*round-robin*) case, is approximately proportional to the number of SZMDs multiplied; with partial parallelism, this relation can be more favorable. The same trade-off principle applies to TDM when communication channels are multiplexed.

In this way, dozens or hundreds of physical SZMDs are logically multiplied into thousands or millions of “virtual” instances, which are coordinated digitally, allowing massive systems to be solved at reduced physical cost.

However, when connecting thousands of SZMDs in cascade to solve coupled differential systems (nonlinear, ordinary, or partial), a classic problem of analog computation arises: *error accumulation*. In the simplest additive case, denoting by e_0 the error of the original input and by δ_k the error introduced by the k -th module, the total error accumulated at the n -th output can be modeled, *in the uncorrelated case*

¹⁶ Also known in the literature as the “Picard–Lindelöf Theorem”.

¹⁷ A metaphorical allusion to the biblical miracle of the multiplication of the loaves and fishes. Just as a few loaves fed thousands of people, a small number of physical SZMDs can be reused (virtualized) over time to process thousands of coupled equations.

that those sources assume, (HIGHAM, 2002, Ch. 3)(WILKINSON, 1994)(NELSON, 1963)

$$e_n = e_0 + \sum_{k=1}^n \delta_k.$$

In systems with feedback or nonlinearities, the propagation can be more severe — for example, multiplicative or exponential, amplified by the Lipschitz constant of the system. A more general modeling assumes propagation gains γ_k at each stage (HIGHAM, 2002, Ch. 3–4)(FIFER, 1961):

$$e_n = e_0 \prod_{k=1}^n (1 + \gamma_k) + \sum_{k=1}^n \delta_k \prod_{j=k+1}^n (1 + \gamma_j), \quad (\text{X.18})$$

which makes evident the need for rigorous control of the errors in each module, especially in the early stages of the cascade.

If all modules operated with the same physical precision, the input signal could fade away or distort completely after passing through many stages. In the SZM, this problem is mitigated by using *hierarchical multiple precision*: the first modules (most critical in signal propagation) should be configured to work with greater stability/precision, while the final stages operate with lower requirements, preventing the error from accumulating destructively.

9. Remark

This capacity to contain error accumulation by means of hierarchically organized multiple precisions represents a fundamental advantage of the SZM over classical analog computers, which failed precisely when trying to solve large-scale problems due to the progressive deterioration of the signal in deep cascades.

10. Summary — Scalable analog-digital hybrid architecture

“The miracle of the multiplication of the SZMDs” — The virtual multiplication of physical SZMDs by means of modern digital computers (with gigantic memory, parallel CPUs, and GPGPUs) allows analog computation to be scaled up to confront very-high-dimensional problems (such as GRNs, metabolic networks, molecular modeling, tissues, biological neural networks, etc.).

In this hybrid architecture, a few real SZMDs constitute the continuous physical core of the computation (fast, reinitializable, and analog), while the digital system acts

continued →

10. Summary — Scalable analog-digital hybrid architecture *(continued)*

as replicator, supervisor, and amplifier, generating thousands or millions of “virtual” instances of these modules. This strategy avoids the prohibitive cost of building thousands of real analog modules and makes it feasible to apply the SZM to massive systems previously unattainable by classical analog computers.

X.9 Application of the Successive Zooms Method to quantum computing

X.9.1 Motivation and general principle

The SZM is a framework of **iterative multi-scale refinement** that decomposes the desired solution into a sum of hierarchical approximations:

$$Q^\star = Q_0 \oplus \Delta Q_1 \oplus \Delta Q_2 \oplus \cdots, \quad (\text{X.19})$$

where each term captures a *residue* not explained by the previous level. The symbol $\oplus$ here denotes the *hierarchical aggregation* of refinements — a sense distinct from the one it has in Subsection X.2.3.1, where it designates the parallel composition of two devices. We propose applying this principle to the quantum domain for **precision amplification** in NISQ processors, without depending exclusively on error-correction codes with large overhead.

X.9.2 Error model and NISQ limitations

Consider an ideal quantum circuit (U) and its noisy implementation ($\hat{U}$). We adopt the decomposition

$$\mathcal{E}(\rho) = (1 - \epsilon)U\rho U^\dagger + \epsilon\Phi(\rho), \quad (\text{X.20})$$

where (Φ) is an effective noise channel and $\epsilon = 1 - \mathcal{F}$ is the complement of the fidelity. The NISQ regime presents typical limitations: (i) temporal decoherence (T_1 , T_2), (ii) gate error, (iii) readout error, and (iv) limited *depth*, leading to the accumulation of stochastic error.

X.9.3 Conceptual integration of the SZM into the quantum domain

The central idea is to insert the SZM as a **quantum–classical metalayer** that operates *between* short quantum executions:

1. A circuit (C_0) is run that produces an estimator ($\hat{y}_0$) for the quantity of interest (e.g., phase, energy, probability).
2. The **residue** (r_1) relative to the target (y) is estimated: $r_1 = y - \hat{y}_0$ (not directly known, but inferred by experimental *design* or by cost functions).
3. A circuit (C_1) is constructed that restricts the state space to the neighborhood compatible with (r_1) (search *zoom*).
4. The process is repeated, generating a sequence ($\hat{y}_k$) and residues (r_{k+1}) until $|r_{k+1}| < \tau$.

The *zoom* can act on variational parameters, circuit topology, or measurement scheme.

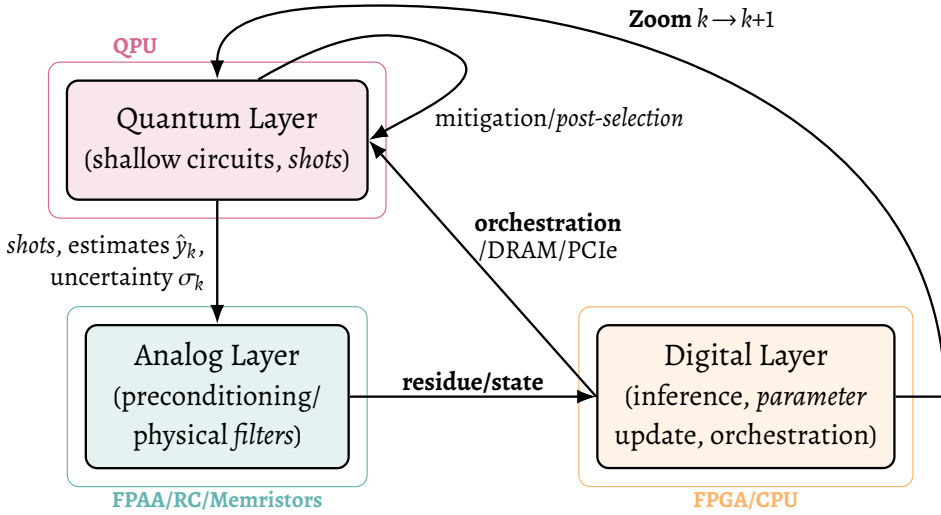


Figure 9 – Tri-hybrid architecture of the SZM quantum–classical metalayer.



X.9.4 Basic mathematical formalism

Let (θ) be the parameter (or vector) to be estimated. Define the cost function

$$\mathcal{J}(\theta) = \mathbb{E}_{\text{samples}} [\ell(y, \hat{y}(\theta))], \quad (\text{X.21})$$

where ($\hat{y}(\theta)$) is the estimator produced by the circuit and (ℓ) is a differentiable loss (or counting statistic). The SZM produces a sequence (θ_k) by hierarchical refinements

$$\theta_{k+1} = \theta_k \oplus \Delta\theta_k, \quad \Delta\theta_k = \arg \min_{\Delta} \mathcal{J}(\theta_k \oplus \Delta). \quad (\text{X.22})$$

X.9.5 Combination of digits (precision amplification)

We can organize the *zooms* so that each iteration aggregates approximately (b) bits:

$$\hat{y} = \hat{y}_0 \oplus 2^{-b}\hat{y}_1 \oplus 2^{-2b}\hat{y}_2 \oplus \dots \quad (\text{X.23})$$

X.9.6 Conclusion

The SZM provides a **unifying framework** for amplifying precision on NISQ hardware, trading depth for controlled re-execution and statistical fusion, with explicit integration of the analog and digital layers as in Figure 9.

The motivation for tri-hybrid architectures is not only computational performance; it is also ontological and epistemological. If biological systems exploit regimes where quantum effects (for example, tunneling) coexist with continuous classical dynamics and with effectively discrete processes, then purely digital simulation tends to fail not only by cost, but by paradigm inadequacy. The discussion of the physical bottleneck of decoherence and of related experimental evidence (including microtubules and anesthesia) is presented in Chapter 8 (Vol. I) — Tunneling of biomolecules, connecting directly molecular biology, physical computability, and modeling requirements.

X.10 The extended analog computer (EAC)

The concept of the *Extended Analog Computer* (EAC) was introduced by Rubel (1993) as a generalization of the *General-Purpose Analog Computer* (GPAC). Later, Mills (2008) developed this model, exploring both formal and experimental aspects of its implementation. The result is a synthesis between an abstract mathematical formulation and practical examples of unconventional analog computation.

X.10.1 Conceptual EAC according to Rubel (1993)

Rubel defined the EAC as a conceptual machine composed of **black boxes** that operate on analytic functions (C^ω).

- **GPAC vs. EAC:** The GPAC deals only with one independent variable (typically time) and initial-value problems of algebraic ODEs, whereas the EAC deals with multiple variables and boundary-value problems for PDEs.
- **Hierarchy of levels:**
 - Level 0: real polynomials.

- Level 1: differentially algebraic analytic functions.
- Level N: progressive constructions from previous levels.
- **Components:** constants, independent variables, adders, multipliers, differentiators, substituters, inverters, set generators, analytic continuation, boundary-value-problem solvers, and restricted limit operators.
- **Extremely Well-Posed (EWP):** small variations of input produce small variations of output on compact sets, ensuring strong determinism and stability.
- **Capabilities:** computing special functions such as $\Gamma(x)$, $\zeta(s)$, and $G(z)$, in addition to solving the Dirichlet problem for the Laplace equation on the disk.

Rubel's motivation was the conviction that the **human brain** is an analog computer more complex than the GPAC can model, and that the EAC would be a step toward that representation.

X.10.2 Implemented EAC according to Mills (2008)

Mills advanced Rubel's proposal by exploring physical and formal implementations of the EAC at Indiana University. His main advances were:

- **Physical implementations:** the use of conductive materials (such as conductive foams and films) capable of solving boundary-value problems for PDEs by analogy with the laws of nature.
- **Explicit vs. Implicit:** a distinction between functions realized explicitly by components (such as adders and constants) and functions realized implicitly by physical laws (such as addition via charge conservation).
- **Configuration, not programming:** unlike digital computers, the EAC is configured by physical and mathematical analogies, and not by sequential algorithms.
- **Δ -digraph:** a formalism that connects **nature, mathematics, and computational architecture**, representing how computation occurs through structural correspondences.
- **Practical examples:** butterfly-wing morphogenesis (reaction-diffusion models), artificial neural networks (XOR), and the damped oscillator (classic in GPAC).

X.10.3 Synthesis

From Rubel and Mills, the EAC can be understood as:

- A **conceptual machine** that generalizes the GPAC, introduced by Rubel.

- A **configurable architecture** with potential physical implementations, formalized by Mills.

While Rubel established the mathematical foundations and the cognitive motivation, Mills advanced toward practical realization, demonstrating that the EAC can serve not only as a theoretical abstraction, but also as an experimental prototype of computation by analogy. This synthesis opens the way for new paradigms, such as the SZM, which can draw inspiration both from Rubel's hierarchical formalization and from Mills's physical implementation and Δ -digraph.

X.11 Partial differential equations (PDEs) and the SZM

The SZM can be interpreted as a conceptual extension of the approach used in the EAC for partial differential equations (PDEs). Whereas the EAC starts from the generalization of the GPAC to multiple variables and boundary-value problems, the SZM adopts the idea of **successive refinements**, or “zooms,” to represent phenomena at multiple scales of description.

X.11.1 Analogy with ODEs and PDEs

The relationship between ODEs and PDEs is maintained in the SZM:

- **ODEs:** deal with the evolution of one variable as a function of a single independent dimension (typically time).
- **PDEs:** deal with functions of several independent variables (time and space), describing more complex phenomena such as diffusion, wave propagation, and fluid dynamics.
- **SZM:** incorporates both, but organizes them into a hierarchy of successive refinements, in which each zoom adds **dimensions, degrees of freedom, or finer resolutions**.

X.11.2 Fundamental blocks of the SZM

Drawing inspiration from the formulation of the EAC, the SZM can be described by analogous “black boxes”:

- **Constants and parameters:** fixed at each zoom level.
- **Zoom Operators:** increase the resolution, subdividing variables and states.
- **Scale Transformers:** reparametrize variables when transitioning between levels.

- **Zoom Couplers:** integrate information between micro and macro levels.
- **Zoom Inverters:** impose top-down restrictions, where emergent phenomena feed back to lower levels.

X.11.3 Well-posedness in the SZM

Just as the EAC is characterized by being *extremely well-posed* (EWP), the SZM requires an analogous criterion:

- Each successive refinement must preserve local stability.
- Small parameter variations at one level cannot generate divergent inconsistencies at the following levels.
- The hierarchy must converge to coherent representations, even when the original differential equation does not admit a classical solution.

X.11.4 Potential applications

The SZM broadens the scope of computability to problems that escape the classical formulation of PDEs:

- Modeling of fields with exotic nonlinearities (e.g., self-confinement of particles).
- Simulations of biological processes at multiple scales (e.g., biomolecular tunneling, gene networks).
- Emergent phenomena in complex networks and chaotic systems.

Thus, the SZM can be seen as a generalization of the logic of the EAC, but with emphasis on the **progressive construction by levels of refinement**, instead of merely extending the differential operators.

X.12 Some limitations of the general-purpose analog computer (GPAC)

The article “Some Mathematical Limitations of the General-Purpose Analog Computer,” by Rubel (1988), deals with the theoretical limitations of the general-purpose analog computer (GPAC — *General-Purpose Analog Computer*). In this section, we will discuss these difficulties from the viewpoints of the three computational approaches: GPAC, Digital, and SZM.

X.12.1 Limitations of the Laplace equation in different computational models

The Laplace equation, $\nabla^2 u = 0$, can be solved in a wide variety of geometries, including disks¹⁸, rectangles, and arbitrary domains with smooth boundaries. Rubel's article shows, however, that there are fundamental limitations when we consider solvability in the abstract model of the *General-Purpose Analog Computer* (GPAC).

X.12.1.1 Digital computers

On digital computers, numerical methods are used, such as finite differences, finite elements, and boundary elements. These methods discretize the domain and produce successively more precise approximations of the solution, converging to the exact solution. Therefore, in the digital paradigm, the Dirichlet problem on the disk is *always* solvable, within the limits of numerical precision.

X.12.1.2 GPAC (general-purpose analog computer)

In the GPAC model, the possible outputs correspond to the class of *differentially algebraic* (DA) functions. Rubel demonstrated that, even when the boundary data of a Dirichlet problem on the disk are DA, the interior solution (obtained via the Poisson integral) may not be DA. Thus, certain fundamental problems of mathematical physics are beyond the reach of the GPAC.

X.12.1.3 SZM

The SZM combines analog and digital resources:

- the analog core accelerates the integration and the handling of stiff systems;
- the digital control manages tolerances, reinitializations, and successive refinements.

This hybrid overcomes the limitation of the GPAC: even if the solution is not DA, the digital component guarantees approximations with arbitrary precision. Thus, the SZM allows the Dirichlet problem (and other PDEs) to be solved in a controllable and efficient way.

Table 6 presents a comparison between the three models.

¹⁸ Here, "disk" means the unit disk $D = \{z \in \mathbb{C} : |z| < 1\}$, that is, the region interior to the circle $|z| = 1$, in the (x, y) plane or in the complex plane. The Dirichlet problem consists in determining a harmonic function $u(x, y)$ within this disk, satisfying $\partial^2 u / \partial x^2 + \partial^2 u / \partial y^2 = 0$ and coinciding on the boundary with the prescribed values.

Table 6 – Comparison of the models and their limitations for solving Dirichlet on the disk

Model	Can it solve Dirichlet on the disk?	Nature of the solution
Digital	Yes	Convergent numerical approximations
GPAC	No (in general)	Limited to DA functions
SZM (hybrid)	Yes	Refinable approximations with arbitrary precision

X.12.2 Fourier series and conjugate series

Rubel showed that a differentially algebraic (DA) Fourier series may exist whose conjugate series is not DA on any interval. In practical terms, the GPAC can produce the original series, but not its conjugate series. This indicates that even simple operations of harmonic analysis do not preserve analog computability.

X.12.2.1 GPAC

In the GPAC model, the operation of obtaining the conjugate series can leave the class of DA functions. Thus, the GPAC cannot, in general, compute the harmonic conjugate of a function, even if it is DA.

X.12.2.2 Digital computers

In digital numerical methods, the computation of the harmonic conjugate is feasible: for example, via the discrete Hilbert transform implemented with FFT, or also by spectral methods. In these cases, controllable and convergent approximations of the conjugate series are obtained, with precision depending on the mesh and the arithmetic used.

X.12.2.3 SZM

In the SZM, this operation can also be performed: the analog component aids in the rapid evaluation of integrals and transforms, while the digital one regulates the error, performs refinements, and corrects approximations. Thus, the SZM is capable of computing conjugate series with arbitrary precision, overcoming the structural limitation of the GPAC. Table 7 presents a comparison between the three models.

Table 7 – Comparison of the models and their limitations for computing the conjugate series

Model	Can it compute the conjugate series?	Nature of the solution
Digital	Yes	Numerical approximations (FFT, Hilbert transform, spectral methods) with controllable error and convergence under refinement
GPAC	No (in general)	The conjugate series can leave the DA class, so it is not computable by the classical analog model
SZM (hybrid)	Yes	Hybrid computation: the analog part accelerates integrals/transforms, the digital part controls tolerances and performs refinement, guaranteeing arbitrary precision

X.12.3 Riesz projection in different computational models

Rubel showed that the Riesz projection — the operation that transforms a general Fourier series into a series of analytic type (keeping only positive coefficients) — can take differentially algebraic (DA) data to results that no longer belong to the DA class. Thus, even if the original series is computable by a GPAC, the projected series may not be.

X.12.3.1 GPAC

The GPAC model is not closed under the Riesz projection. There are explicit counterexamples in which the projection of a DA series results in a non-DA series. This shows that certain classical harmonic transformations cannot be performed by general-purpose analog computers.

X.12.3.2 Digital computers

In digital numerical methods, the Riesz projection is implementable in a stable way. For example:

- via FFT, zeroing the negative coefficients and reconstructing the series;
- via quadratures or spectral filtering in periodic domains.

These methods provide convergent and controllable approximations, within machine precision.

X.12.3.3 SZM

In the SZM, the Riesz projection can be performed in a hybrid flow:

- the digital component identifies and eliminates the unwanted coefficients;
- the analog component can accelerate continuous reconstructions or related integrals;
- an iterative refinement loop guarantees arbitrary precision.

Thus, the SZM overcomes the structural limitation of the GPAC, guaranteeing the practical computability of the Riesz projection. Table 8 presents a comparison between the three models.

Table 8 – Comparison of the models and their limitations for performing the Riesz projection

Model	Can it perform the Riesz projection?	Nature of the solution
Digital	Yes	Implementable by FFT zeroing negative coefficients, or by spectral quadratures, with controllable error
GPAC	No (in general)	The projection can transform DA data into non-DA functions, beyond the reach of the GPAC
SZM (hybrid)	Yes	Hybrid flow: digital identifies coefficients, analog accelerates continuous reconstructions, and digital refinements ensure arbitrary precision

X.12.4 Riesz–Herglotz (inner–outer) and Blaschke factorization

A classical result of complex analysis states that any bounded analytic function on the unit disk can be factored as a product of an *inner* function and an *outer* function. In addition, the *inner* part can be decomposed into a Blaschke product and a singular factor. Rubel showed that, even when the original function is differentially algebraic (DA), the factors obtained may not be DA.

X.12.4.1 GPAC

In the GPAC model, which only generates DA functions, the Riesz–Herglotz factorization represents a serious limitation. DA functions can have factors (inner or outer) that do not belong to the DA class, so the GPAC cannot compute such components. This shows that central structural operations of complex analysis are not preserved.

X.12.4.2 Digital computers

In the digital paradigm, factorization is tractable by numerical methods:

- the zeros of the function can be estimated by iterative methods (Newton, regula falsi, etc.), enabling the construction of the Blaschke product;
- the outer part can be obtained by discretization of the logarithmic integral associated with the function.

Although there are inevitable numerical errors, the approximations converge under mesh refinement and increased arithmetic precision.

X.12.4.3 SZM

In the SZM, factorization can be implemented in a hybrid way:

- the digital search identifies and refines the zeros of the function with adjustable tolerance;
- the analog core aids in computations of continuous integrals and real-time multiplications;
- the digital refinement loop guarantees arbitrary precision in the recomposition of the factors.

Thus, the SZM is capable of performing inner–outer factorization and constructing Blaschke products, overcoming the theoretical limitations of the GPAC. Table 9 presents a comparison between the three models.

Table 9 – Comparison of the models and their limitations for performing factorization

Model	Can it perform factorization?	Nature of the solution
Digital	Yes	Zeros estimated numerically (Newton, regula falsi) and outer obtained via discretized logarithmic integral
GPAC	No (in general)	Factorization can produce non-DA factors, infeasible for the classical analog model
SZM (hybrid)	Yes	Digital search locates zeros, analog aids integrals and multiplications, digital guarantees refinement and arbitrary precision

X.12.5 Cousin problem (DA version)

The Cousin problem, in its classical formulation, consists in decomposing a function into parts that extend analytically to distinct domains. In a simplified version, given a function $u(x)$ on the real line, one wishes to write it as

$$u(x) = u^+(x) - u^-(x),$$

where u^+ extends holomorphically to the upper half-plane and u^- to the lower half-plane. Rubel showed that, even when $u(x)$ is differentially algebraic (DA), this decomposition may fail to preserve the DA class.

X.12.5.1 GPAC

In the GPAC model, which only generates DA functions, the solution of the Cousin problem is not guaranteed. Counterexamples show that, even if the initial function is DA, the parts u^+ and u^- may not be DA. Thus, the GPAC cannot, in general, perform this decomposition.

X.12.5.2 Digital computers

In digital methods, the problem can be treated approximately using:

- Cauchy and Plemelj integral formulas in suitable discretizations;
- boundary element methods in one dimension (BEM), to represent analytic extensions.

These techniques allow convergent approximations of u^+ and u^- with control of the numerical error.

X.12.5.3 SZM

In the SZM, the decomposition can be performed in a hybrid flow:

- the analog component accelerates singular integrals and continuous transforms;
- the digital control manages regularizations, subdivisions, and zooms in critical regions;
- arbitrary precision is obtained by iterating the process.

Thus, the SZM overcomes the structural limitation of the GPAC and enables the practical construction of the decomposition of the Cousin problem. Table 10 presents a comparison between the three models.

Table 10 – Comparison of the models and their limitations for solving Cousin

Model	Can it solve Cousin (DA version)?	Nature of the solution
Digital	Yes	Treated numerically by Cauchy/Plemelj formulas and 1D BEM, with convergent approximations
GPAC	No (in general)	Even with a DA initial function, u^+ and u^- may not be DA
SZM (hybrid)	Yes	Analog core accelerates singular integrals, digital regulates subdivisions and refinements, guaranteeing arbitrary precision

X.12.6 Dirichlet problem and Poisson integral

The Dirichlet problem for the Laplace equation on the disk consists in determining a harmonic function $u(x, y)$ in the interior of the unit disk, given a set of boundary values $u(\theta)$ on its edge. The classical solution is provided by the Poisson integral:

$$u(re^{i\theta}) = \frac{1}{2\pi} \int_0^{2\pi} P_r(\theta - \phi) f(e^{i\phi}) d\phi,$$

where P_r is the Poisson kernel and f represents the boundary data. Rubel showed that, even when f is a differentially algebraic (DA) function, the solution u may not be DA. This means that the GPAC cannot, in general, solve the Dirichlet problem via the Poisson integral.

X.12.6.1 GPAC

In the GPAC model, the set of generated functions is restricted to the DA ones. Since the Poisson integral can produce functions outside this class, the GPAC fails to solve certain Dirichlet problems, even in a domain as simple as the unit disk.

X.12.6.2 Digital computers

On digital computers, the Dirichlet problem is widely tractable:

- finite difference (FD) and finite element (FEM) methods approximate the solution by discretizing the domain;
- in the specific case of the disk, the Poisson integral can be evaluated numerically by quadratures.

These techniques provide approximate solutions that converge to the real solution when the discretization is refined.

X.12.6.3 SZM

In the SZM, the solution of the Dirichlet problem is enabled by the hybrid combination:

- the analog core can rapidly compute continuous integrals or simulate harmonic relaxation;
- the digital control regulates tolerances, applies local refinements, and guarantees arbitrary precision;
- successive zooms allow resources to be concentrated in regions critical to the solution.

Thus, the SZM overcomes the structural limitation of the GPAC and solves the Dirichlet problem with efficiency and control over precision. Table 11 presents a comparison between the three models.

Table 11 – Comparison of the models and their limitations for solving Dirichlet

Model	Can it solve Dirichlet (Poisson)?	Nature of the solution
Digital	Yes	Approximate solution via FEM, FD, BEM, or quadratures, converging under refinement
GPAC	No (in general)	The Poisson integral can generate functions outside the DA class
SZM (hybrid)	Yes	Analog performs fast integrals or relaxations, digital guarantees refinements and arbitrary precision

X.12.7 Calculus of variations (multidimensional)

The calculus of variations seeks to find functions that minimize integral functionals. In one dimension, when the solution is analytic, Rubel and collaborators showed that the problem can be solved by a GPAC. However, Rubel demonstrated that, in multidimensional cases, such as the minimization of the Dirichlet integral,

$$\iint \left((u_x)^2 + (u_y)^2 \right) dx dy,$$

subject to boundary conditions, the solution may not be differentially algebraic (DA). This means that the GPAC cannot, in general, solve these problems.

X.12.7.1 GPAC

The GPAC model, restricted to DA functions, does not preserve this class when dealing with variational problems in multiple dimensions. The minimization of the Dirichlet integral can generate non-DA solutions, making the problem intractable in this paradigm.

X.12.7.2 Digital computers

In digital methods, the calculus of variations is widely treated by techniques such as:

- the finite element method (FEM), which formulates the variational problem in terms of approximation spaces;
- Galerkin discretizations, which convert the problem into an algebraic system solvable numerically.

These methods produce approximate solutions that converge under mesh refinement and increasing order of the shape functions.

X.12.7.3 SZM

In the SZM, multidimensional variational computation can be treated in a hybrid way:

- the digital component manages the discrete formulation and the adaptive refinements;
- the analog core can accelerate the evaluation of local integrals and simulate continuous relaxations;
- successive zooms allow resources to be concentrated in critical regions to minimize the functional with arbitrary precision.

Thus, the SZM overcomes the structural limitation of the GPAC, offering a practical method for solving multidimensional variational problems. Table 12 presents a comparison between the three models.

Table 12 – Comparison of the models and their limitations for solving the calculus of variations

Model	Can it solve the calculus of variations (multi-D)?	Nature of the solution
Digital	Yes	FEM and Galerkin methods discretize the problem, with assured convergence
GPAC	No (in general)	The variational solution may not be DA, infeasible on the GPAC
SZM (hybrid)	Yes	Digital control manages discretization, analog accelerates integrals, successive refinements guarantee arbitrary precision

X.12.8 Integrals with general kernels

Rubel discussed the question of for which kernels $K(x, y)$ the transformation

$$f(x) = \int K(x, y)g(y)dy$$

preserves the class of differentially algebraic (DA) functions, when g is DA. He showed that, although some particular cases preserve the DA class (such as certain separable kernels), fundamental classical kernels — such as those of Poisson and Cauchy — can send DA data to functions that are not DA. This means that the GPAC is not closed under general integral operations.

X.12.8.1 GPAC

In the GPAC model, integrals with general kernels do not, in general, preserve the DA class. Important counterexamples are the Poisson kernel (in the Dirichlet problem) and the Cauchy kernel (in the Cauchy integral), which produce solutions outside the DA class, even with DA initial data.

X.12.8.2 Digital computers

In digital methods, integrals with general kernels are routinely tractable:

- quadrature techniques (simple, adaptive, or spectral) allow integrals to be evaluated with error control;
- boundary element methods (BEM) naturally handle Poisson and Cauchy integrals in boundary problems.

The solution is approximate, but converges to the exact solution under suitable refinement.

X.12.8.3 SZM

In the SZM, integrals with general kernels can be solved by hybrid cooperation:

- the analog core accelerates the computation of continuous integrals;
- the digital control applies regularizations, subdivisions, and adaptive zooms;
- arbitrary precision is guaranteed by successive iterations and numerical control.

Thus, the SZM circumvents the structural limitation of the GPAC and allows integrals with classical kernels to be solved in a practical and precise way. Table 13 presents a comparison between the three models.

Table 13 – Comparison of the models and their limitations for solving general integrals

Model	Can it solve general integrals?	Nature of the solution
Digital	Yes	Adaptive quadratures and spectral methods handle general kernels with controllable error
GPAC	No (in general)	Only very specific classes of kernels preserve DA; kernels such as Poisson and Cauchy are not feasible
SZM (hybrid)	Yes	Analog handles continuous integrals, digital regulates regularization and stability, guaranteeing arbitrary precision

X.13 Solving PDEs using FPGAs

The work of Thai and Tung (2019) presents a practical approach for solving partial differential equations (PDEs) by means of reconfigurable digital architectures, specifically using *Field Programmable Gate Arrays* (FPGAs) combined with *cellular neural networks* (CNNs). This proposal highlights the potential of parallel hardware to handle problems traditionally associated with continuous analog computation.

X.13.1 Motivation and context

PDEs such as those of Navier–Stokes, Burgers, and Saint Venant play a central role in the modeling of fluids, waves, and nonlinear dynamical systems. The numerical

solution of these equations on conventional digital computers is usually limited by:

- high computational costs;
- the need for fine discretizations for greater precision;
- difficulties in achieving real-time performance.

Thai and Tung's proposal is to exploit FPGAs, which offer massive parallelism and reconfigurability, to accelerate the solution of these equations.

X.13.2 Cellular neural networks (CNNs)

CNNs form the basis of the implementation. Each cell represents a small part of the spatial domain of the equation and interacts only with nearby neighbors, mimicking the local structure of the discretized PDEs.

- **Local structure:** each cell updates its state as a function of its neighbors, which allows natural parallelization.
- **Physical analogies:** processes such as diffusion and wave propagation emerge from the collective dynamics of the cells.
- **Flexibility:** different local kernels can implement various PDEs.

X.13.3 FPGA implementation

The implementation exploits the parallel nature of CNNs:

- The domain is divided into subdomains processed in parallel on the chip.
- Buffers and registers handle the communication at the boundaries between blocks.
- The pipeline is optimized to reduce latencies and maximize throughput.
- The hardware is reconfigurable, allowing the same structure to be adapted to different types of PDEs.

X.13.4 Case studies

The authors describe concrete examples:

- **Burgers equation:** used as a prototype of a nonlinear diffusion equation.
- **Navier–Stokes equations:** applied to the dynamics of incompressible fluids¹⁹.

¹⁹ In the general case, the Navier–Stokes equations can be formulated for compressible fluids, but this requires including the continuity equation in its complete form, the energy equation, and an equation of state (such as that of ideal gases). This set results in a more complex system of coupled equations.

- **Saint Venant equations:** used in hydrodynamic modeling.

In all cases, the FPGA showed competitive and scalable performance.

X.13.5 Results and contributions

- Proof of concept that PDEs can be solved in real time on dedicated hardware.
- Demonstration of the feasibility of CNNs as a hybrid alternative between physical analogy and digital computation.
- Provision of a scalable architecture, capable of handling boundary problems and extensive meshes.

X.13.6 Implications

This article establishes a bridge between Rubel's vision of the EAC (an idealized analog computer) and practical digital architectures. Although it is not an analog computer in the strict sense, the FPGA with CNNs reproduces the **local logic** of PDEs, becoming an example of computation **configured by analogy** rather than programmed by sequential algorithms.

X.14 From differential equations to successive zoom architectures

The trajectory connecting the *General-Purpose Analog Computer* (GPAC), the *Extended Analog Computer* (EAC), the digital FPGA implementations, and, finally, the SZM, reveals an evolution in the way of dealing with ordinary (ODEs) and partial (PDEs) differential equations. This progression goes from mathematical abstraction to physical implementation and, finally, to the formulation of new computational paradigms.

X.14.1 Conceptual EAC (Rubel, 1993)

Rubel (1993) proposed the EAC as a generalization of the GPAC.

- It extends analog computation to multiple independent variables.
- It solves not only initial-value problems (ODEs), but also boundary-value problems for PDEs.
- It has a hierarchy of levels that generates increasingly complex functions from polynomials.

- It is characterized by being *extremely well-posed*, guaranteeing determinism and stability.
- It is capable of computing special functions such as $\Gamma(x)$ and $\zeta(s)$.

X.14.2 Implemented EAC (Mills, 2008)

Mills advanced on Rubel's model by exploring experimental prototypes:

- The use of conductive materials (foams, films) to solve boundary-value problems via physical analogy.
- The introduction of the Δ -digraph formalism, which connects nature, mathematics, and computational architecture.
- A distinction between explicit functions (realized by components) and implicit ones (realized by the physical laws themselves).
- Practical examples: butterfly-wing morphogenesis, XOR neural networks, and the damped oscillator.

X.14.3 Digital FPGA implementations (Thai & Tung, 2019)

With the advance of digital technology, Thai and Tung demonstrated the solution of complex PDEs in *Field Programmable Gate Arrays* (FPGAs) through *cellular neural networks* (CNNs).

- Parallelization and discretization strategies allow equations such as Burgers, Navier–Stokes, and Saint Venant to be solved.
- The architecture divides space into local subdomains, with boundary control and buffers.
- CNNs approximate the behavior of continuous systems by means of discrete local interactions.
- The results show that it is possible to achieve real-time computations with hardware efficiency.

X.14.4 The SZM

Drawing inspiration from the logic of the EAC and from the discrete FPGA implementations, the SZM proposes a new paradigm:

- **Successive refinement:** each “zoom” increases the resolution of the description of the phenomenon, adding dimensions, degrees of freedom, or internal structures.

- **Fundamental blocks:** zoom operators, scale transformers, level couplers, and zoom inverters.
- **Well-posed problem:** each refinement must preserve local stability and convergence between levels.
- **Reach:** capable of modeling problems that escape the classical formulation of PDEs, including fields with exotic nonlinearities, biomolecular tunneling, and complex networks.

X.14.5 Historical synthesis

- **Rubel (1993):** conceived the EAC as a universal mathematical machine for analytic functions and PDEs.
- **Mills (2008):** demonstrated experimental implementations with physical materials, highlighting computation by analogy.
- **Thai & Tung (2019):** brought the concept to parallel digital architectures in FPGAs, with CNNs capable of solving PDEs in real time.
- **SZM:** proposes the integration of these advances in a hierarchical and iterative approach, broadening the scope to phenomena beyond classical equability.

Thus, the SZM emerges as the natural heir of this line of research, combining the **conceptual universality of the EAC**, the **physical implementation of Mills**, and the **digital efficiency of FPGA/CNNs**, in an architecture of successive zooms oriented to the solution of complex, multiscale problems.

X.15 Continuous blocks versus discrete meshes in solving PDEs

The comparison between Mills's (2008) proposal, founded on the idea of the *Extended Analog Computer* (EAC), and Thai & Tung's (2019) implementation on FPGAs, reveals two radically distinct approaches to the solution of partial differential equations (PDEs).

X.15.1 Mills's continuous approach

In Mills's formulation, inspired by Rubel, the EAC is conceived as a universal analog machine:

- Computation is performed by means of **continuous functional blocks**, such as soap films (minimal-surface equation), conductive media (Laplace equation), or physical networks.

- PDEs are solved by **direct analogy with physical laws**, in which the domain is represented as a whole, without the need for discretization into numerical meshes.
- The process is essentially **continuous**, preserving the mathematical structure underlying the problem.

X.15.2 Thai & Tung's discrete approach

In contrast, Thai & Tung (2019) use a massively parallel digital approach:

- The PDEs are **discretized on a mesh**, and each cell of the network represents a point of the spatial domain.
- The method is based on **cellular neural networks (CNNs)**, in which each cell interacts only with its immediate neighbors.
- The parallelism of the FPGA allows **real-time** simulations, exploiting the local and simultaneous updating of thousands of cells.

X.15.3 Synthesis and implications

We thus have two contrasting paradigms:

Mills: solution by **continuous physical blocks**, supported by direct analogies with natural systems.

Thai & Tung: solution by **discrete meshes**, with parallel digital updating on FPGAs.

This difference is particularly relevant to the formulation of the SZM, which can be conceived as a **conceptual bridge between the continuous and the discrete**. The SZM dialogues with the analog ideal of the EAC (a global, continuous representation of the domain) and, at the same time, with the computational practicality of massive discretization on FPGAs. In this sense, the SZM offers a natural framework for integrating multiple levels of representation — from the continuous to the discrete, from the macro to the micro — through successive refinements.

X.16 A survey on analog computation for domain-specific accelerators

The article by Belostotski et al. (2025) provides a comprehensive view of the resurgence of analog computation, with a focus on domain-specific accelerators. The

survey highlights how analog architectures can offer significant gains in energy efficiency and parallelism, in contrast to the inherent limitations of traditional digital systems.

X.16.1 What “domain-specific” means

In the context of the article, **domain-specific** refers to a restricted field of application or to a particular class of mathematical/physical problems for which the accelerator is optimized. Instead of seeking the universality typical of general-purpose digital computers, or even of generalized analog architectures (such as the GPAC or the EAC), these accelerators are designed “tailor-made” to obtain maximum performance and efficiency in a delimited application. Examples include:

- **Linear algebra:** analog solvers specialized in linear systems $Ax = b$.
- **Partial differential equations (PDEs):** RC/LC circuits, metamaterials, or photonic architectures that solve diffusion, wave propagation, or Navier–Stokes equations.
- **Combinatorial optimization:** Ising machines and analog annealers.
- **Machine learning:** neuromorphic accelerators and *In-Memory Computing* (IMC) architectures for matrix multiplications.

This strategy sacrifices universality in favor of substantial gains in energy efficiency and massive performance.

X.16.2 Motivations and advantages

The authors emphasize the advantages of the analog paradigm:

- **Energy efficiency:** analog architectures can achieve up to a thousand times more efficiency than equivalent digital implementations in some applications.
- **Physical parallelism:** the computation occurs simultaneously at different points of the domain, without the need for sequential clock cycles.
- **Natural approximation of continuous systems:** physical phenomena can be exploited directly as solvers of differential equations.

X.16.3 Applications covered

The survey covers several areas of application:

- **In-memory computing (IMC):** integration of processing directly into memory arrays.

- **Ising machines:** aimed at combinatorial optimization problems.
- **Analog PDE and linear-algebra solvers:** based on waves, metasurfaces, and RC or LC circuits.
- **Neuromorphic processors:** architectures that mimic neural connectivity.
- **Machine learning (ML) accelerators:** exploiting large-scale analog multiplications.
- **Systems inspired by quantum physics:** the use of optical, acoustic, and meta-material devices for computation by analogy.

X.16.4 Solutions for PDEs

Particular attention is given to techniques for partial differential equations:

- **Wave-propagation-based methods:** they use electromagnetic metasurfaces, acoustic structures, and photonic arrays.
- **SDTC (spatially discrete, time-continuous) architectures:** they implement continuous differential operators by means of analog all-pass filters.
- **Discrete-time analog circuits:** with capacitors and switched currents, reconciling precision and robustness.
- **Performance:** reported gains of up to 1000× in energy efficiency and 420× in speed, compared with digital FDTD.

X.16.5 Challenges and limitations

Despite the advances, the article stresses obstacles to large-scale adoption:

- **Precision:** limitations in accuracy due to noise and fabrication variations.
- **Scalability:** difficulty in designing and interconnecting complex analog systems.
- **Software tools:** the lack of a mature ecosystem comparable to that existing for digital design (HDLs, EDA, etc.).

X.16.6 Implications for the SZM

The survey confirms that analog computation is expanding, exploring both classical paradigms (RC/LC, analog neural networks) and emerging ones (photonics, acoustics, metasurfaces). For the SZM, these advances are particularly relevant:

- The SZM can be interpreted as a **unifying structure**, capable of integrating different implementation styles (continuous analog, hybrid, discretized, photonic).
- The physical parallelism and multiscale refinement discussed in the survey dialogue directly with the idea of **successive zooms**.
- The review article offers a **contemporary anchor**, positioning the SZM within an active and growing line of research.

X.17 Hybrid analog-digital solution of nonlinear PDEs by Huang et al. (2017)

The central proposal of Huang et al. (2017) is to accelerate the solution of nonlinear partial differential equations (PDEs) by means of a *programmable analog accelerator* coupled to a digital host. The objective is to combine the robustness and scalability of digital computation with the speed and energy efficiency of continuous-time analog computation.

X.17.1 Idea of the solution

The weak point of traditional digital methods (classical Newton and damped variants) is the strong dependence on the *initial guess* and on the choice of step. This leads to high computational cost or even non-convergence. The analog accelerator solves this problem by formulating the Newton method as a continuous-time ordinary differential equation (ODE). Thus:

- The circuit implements integration, evaluation of the nonlinear function, and computation of the Jacobian in continuous time.
- The continuous dynamics drastically reduces the sensitivity to initial conditions.
- The analog solution, even when approximate (RMS error in the range of 5%), falls within the quadratic convergence basin of the digital solver.

The final flow is hybrid: the analog provides a robust and fast initial guess; the digital refines to high precision.

X.17.2 Discretization

The discretization process is kept on the digital host:

1. **Space:** discretization by second-order central finite differences.
2. **Time:** advance by Crank–Nicolson (implicit, second order).
3. At each time step, a nonlinear system of algebraic equations $F(u) = 0$ is generated.

The stencil and its Jacobian are then loaded into the analog accelerator.

X.17.3 Hardware architecture

The prototype (TSMC 65 nm, 3.7 mm × 3.9 mm) has:

- Tiles with integrators, multipliers, DACs, ADCs, and a programmable cross-bar.
- Neighbor-to-neighbor connectivity, matching the patterns of structured PDE meshes.
- A C++ object-oriented digital interface to configure equations and Jacobians, define initial conditions, and execute.

Power consumption is extremely low (order of mW), and the convergence time tends to be approximately constant with the size of the problem up to area limits (16×16 nodes in the prototype).

X.17.4 Limitations

- Precision limited by ADCs, process variations, and mismatch (RMS error of 5.38%).
- Unstructured meshes would require frequent reconfiguration.
- Transcendental nonlinearities (e.g., e^u , $\sin u$) require approximations.

X.17.5 Integration with the SZM

The SZM can directly incorporate this paradigm:

- The **continuous analog Newton** functions as an *initial-approximation module*, reducing the search space of nonlinear solutions.
- The digital refinement phase can be absorbed or reconfigured according to the principles of the SZM, allowing greater hybridization between analog, digital, and possibly quantum computation.
- The approach shows how *structured meshes and classical discretizations* can be handled efficiently within an SZM architecture, while admitting the replacement of traditional integrations by analog dynamics.

Thus, the work of Huang et al. demonstrates a practical path for the integration of analog and digital computation in PDEs, and provides a concrete example of how the SZM can absorb analog acceleration techniques to deal with nonlinearities and regimes of high complexity.

X.17.6 Comparison between Mills (2008), Thai and Tung (2019) and Huang et al. (2017)

To situate the proposal of Huang et al. (2017) within the line of research on analog and hybrid computation, it is instructive to compare it with the approaches of Mills (2008) and Thai and Tung (2019).

Mills (2008)

The proposal of the *Extended Analog Computer* (EAC) is to map the PDE directly onto the physical domain. Conductive foams, films, and Łukasiewicz logic materialize the equations, and the solution emerges by direct physical analogy. There is no numerical discretization: the mesh is replaced by the continuity of the material. It is an approach of pure analog computation.

Thai and Tung (2019)

In this line, the PDE is converted into a discrete mesh and implemented on an FPGA using cellular neural networks (CNNs). Each cell corresponds to a point of the mesh, and the temporal evolution occurs by local update rules. It is an essentially digital and parallel approach, which depends on explicit discretization.

Huang et al. (2017)

Unlike Mills, the authors do not solve the entire PDE by physical analogy. Space and time are discretized on the digital host (central finite differences and Crank–Nicolson). The hardest part — the resulting nonlinear system at each time step — is solved by a continuous-time analog accelerator, which implements the continuous Newton. Thus, the analog module provides a robust initial guess, accelerating the convergence of the digital solver.

Synthesis.

The proposal of Huang et al. (2017) is closer to Thai and Tung (2019) than to Mills (2008), since it maintains the traditional digital discretization and uses the analog only as an *auxiliary kernel*. In contrast, Mills (2008) represents the radically analog route, while Thai and Tung (2019) remain in the pure digital. The work of Huang et al. (2017) shows an intermediate path, exploiting hybridization as a means of obtaining practical gains in robustness and efficiency.

X.18 Stochastic (Monte Carlo) method for PDEs on analog computers

The stochastic method, commonly called the *Monte Carlo Method*, exploits the correspondence between *random processes* (e.g., Brownian motions or random walks) and solutions of boundary-value problems for elliptic and parabolic PDEs. On analog computers, this correspondence is implemented physically by random excitations and blocks that perform integrations and filterings, so that the statistics of the trajectories provide pointwise estimates of the solution.

Principle.

Instead of solving the deterministic PDE directly, the Dirichlet/Neumann problem is modeled as a diffusion process with absorbing/reflecting boundaries. The solution $u(x)$ at an interior point x is estimated as the average of boundary values sampled at the end of multiple random trajectories that start from x and evolve to the boundary, according to a *random walk* scheme.

Random walk.

One typically uses the *Discrete Random Walk with Absorbing Boundary* or *grid-free* variants such as *Walk on Spheres* (WoS), in which the particle jumps between maximally inscribed spheres in the domain until it reaches (or approaches) the boundary. Mixed boundary conditions are handled by rules of absorption (Dirichlet) and reflection (Neumann) during the simulation.

Solution and convergence.

The *pointwise* estimate $\hat{u}(x)$ follows from the sample average of N trajectories: $\hat{u}(x) = \frac{1}{N} \sum_{k=1}^N g(X_k)$, where g evaluates the boundary condition at the exit point X_k . By the laws of large numbers, $\hat{u}(x)$ converges to the analytic solution $u(x)$ as $N \rightarrow \infty$. Classical results (e.g., Petrowsky for generalized Dirichlet problems) ensure convergence under usual regularity conditions.

Applicability and advantages.

The method is naturally parallel, independent of a volumetric mesh (especially in WoS variants), and handles complex geometries well. It is especially attractive for diffusion/conduction problems and for scenarios in which local estimates at a few points of the domain are sufficient. In analog and hybrid (analog+digital) implementations, the cost grows primarily with the desired precision (number of samples), and not with the global spatial discretization.

Notes on analog/hybrid implementation.

On classical analog computers, noise sources and integration circuits perform the generation of trajectories; event counting/statistics provides the averages. In modern hybrid architectures, the analog performs continuous blocks (generation/filtering) while the digital controls sampling, boundary decisions, and accumulation of statistics. Variants with memristors and reconfigurable arrays also support related SDEs.

X.19 Digital differential analyzers (DDA)

Digital Differential Analyzers (DDA) are **digital integrating computers** designed to solve differential equations and other continuous mathematical problems by means of the **successive summation of discrete increments**. Unlike general-purpose digital computers, the DDA reduces all operations to a single fundamental process: **incremental digital integration**, performed by additions and subtractions in registers that accumulate values in fixed time steps.

Table 14 – Comparison between classical and modern DDAs.

Aspect	Classical DDAs (1940–1960)	Modern DDAs (FPGA and variants)
Technology	Discrete circuits, relays, vacuum tubes or early transistors; magnetic-drum or ferrite memory.	FPGAs, programmable logic, advanced numerical architectures in fixed or floating point.
Speed	Hundreds or thousands of integrations per second.	Much higher, limited by the clock frequency of the FPGA/SoC (millions of operations per second).
Flexibility	Fixed architecture; difficult reconfiguration.	Highly reconfigurable; alteration of algorithms via reprogramming of the FPGA.
Cost	High; dedicated and specialized hardware.	Relatively low; implementations on general-purpose reconfigurable hardware platforms.
Applications	Flight control, ballistics, missile navigation, industrial automation.	Scientific simulation, teaching, real-time research, prototyping of dynamical systems.

Main characteristics

- **Hybrid nature:** combines the methodology of analog computers (direct solution of ODEs) with digital implementation (adders, registers).
- **Incremental representation:** continuous variables are represented by streams of discrete increments, conceptually equivalent to delta modulation.
- **Typical applications:** ballistics, inertial navigation, missile control, flight simulation, and industrial automation.

The first DDA implemented in hardware was the **MADDIDA** (*Magnetic Drum Digital Differential Analyzer*), developed by the Northrop Aircraft Corporation between 1946 and 1949. Its construction demonstrated the feasibility of replacing analog differential analyzers with specialized digital machines. In the following decades, variants emerged such as the DDA-20 (Litton Industries, USA), the TEDDA (Japan), and various high-speed parallel DDAs in the USSR, the USA, and Japan.

Although the classical DDAs emerged between the 1940s and 1960s, their concept remains influential in modern implementations of specialized hardware, such as FPGAs and reconfigurable accelerators (FASIH et al., 2009; BARTEL; KORCH, 2023). Table 14 summarizes the differences between the generations.

X.19.1 Limitations of DDAs for the problems treated in this book

Despite their conceptual elegance and suitability for non-*stiff* real-time problems, DDAs present fundamental limitations that make them inadequate for the classes of problems central to this book:

- **Fixed integration step:** the DDA uses uniform time increments. For *stiff* ODEs, it would be necessary to reduce the step to extremely small values, making efficiency infeasible.
- **Absence of implicit methods:** integration in DDAs is analogous to the explicit Euler method. *Stiff* equations demand implicit methods (Backward Euler, BDF) to guarantee numerical stability.
- **Error accumulation:** each discrete increment adds quantization errors (hardware) or rounding errors (software). In *stiff* problems with multiple time scales, these errors amplify rapidly.
- **Frequency limitation:** in hardware, the clock frequency restricts the update rate, insufficient for *stiff* systems with very fast dynamics.

Consequently, DDAs are not capable of solving the main problems treated in this book — gene regulatory networks (GRNs), metabolic networks, molecular dynamics, agent-based tissue simulations, strongly oscillatory PDEs, and *mesh-free* simulations with nonlinear interactions — all of which require adaptive, implicit, and multiscale methods. DDAs remain relevant as historical landmarks and for simple real-time differential problems, but they do not offer the resources necessary for the computational challenges that motivate the SZM.

X.20 Analysis of the MacLennan (2007) article

This section analyzes the review report of MacLennan (2007), compares GPACs and digital computers, and connects the conclusions to the SZM.

X.20.1 Advantages and disadvantages of analog computers relative to digital ones

Chapter 3 of MacLennan (2007) details the advantages and disadvantages of analog computers compared with digital ones (SMALL, 1993; SMALL, 2001; MINDELL, 2002).

Advantages of analog computers

- **Speed and parallelism:** electronic analog computers could be several orders of magnitude faster than the digital ones of the early decades. Whereas digital computers processed operations sequentially, analog ones worked in parallel, maintaining speed even on larger problems (SMALL, 2001).
- **Suitability for low precision:** many engineering problems used approximate experimental data and incomplete models. In these situations, very high digital precision was unnecessary and even misleading. Analog computers offered more realistic solutions, close to physical reality (MINDELL, 2002).
- **Practical exploration:** they allowed parameters to be adjusted in real time and effects to be observed directly, favoring intuition and practical knowledge (MINDELL, 2002).
- **Hybrid applications:** in the 1950s–60s, they were widely employed in hybrid systems (digital + analog), combining the speed for closed-loop control simulation with digital precision in missile trajectories (SMALL, 1993).

Disadvantages of analog computers

- **Limited and expensive precision:** in general, they worked with 3 to 4 digits of precision. Raising this precision was very expensive, since it required very-high-quality components. Digital computers, on the other hand, increased precision incrementally and cheaply (SMALL, 2001).
- **Social and academic perception:** the written programming and the precision of digital computers were associated with mathematics and science, whereas analog computers, linked to engineering practice, were seen as of “lower status” (SMALL, 2001).
- **Restricted applicability:** analog computers were better suited to continuous variables; digital ones proved versatile for business applications and discrete data processing, which attracted greater industrial investment (MINDELL, 2002).
- **Technological obsolescence:** the rapid evolution of digital electronics (cheapening, miniaturization, a vibrant industry) displaced hybrids and pure analog computers from the 1970s onward (SMALL, 1993).

In summary, analog computers had speed, parallelism, and suitability for low-precision problems as their assets, but lost ground because they were expensive to manufacture at high precision, had restricted reach, and were associated with an

image of being “less scientific.” Digital computers, in turn, benefited from technological advances, versatility, and association with market applications, consolidating their hegemony (SMALL, 2001; MINDELL, 2002).

X.20.2 The SZM in the face of the advantages and disadvantages of analog computers

In considering the comparisons in Chapter 3 of MacLennan’s report, the SZM presents itself as a proposal capable of capitalizing on the strengths of analog computation, while at the same time offering solutions to its historical weaknesses.

Exploiting the advantages

- **Speed and parallelism:** the SZM, by employing analog circuits in hierarchical layers of *zooms*, maintains the parallel exploration of multiple temporal and spatial scales, benefiting from the intrinsic speed of continuous devices.
- **Suitability for low precision:** just like analog computers, the SZM recognizes that many biological and physical phenomena do not require arbitrary precision. It uses robust continuous approximations, but allows digital corrections where accuracy is critical.
- **Practical and interactive exploration:** the hierarchical structure of the SZM favors parametric adjustments in real time, allowing complex dynamical regimes to be explored and emergent behavior to be observed at different levels of granularity.
- **Hybrid integration:** the SZM is not limited to pure analog; it combines digital modules for storage, control, and reconfiguration, forming a hybrid system in which each paradigm is used for what it does best.

Overcoming the disadvantages

- **Limited and expensive precision:** by delegating to the digital layers the responsibility for correction, verification, and storage, the SZM avoids the need for very-high-precision analog components, reducing costs and increasing reliability.
- **Social and academic perception:** the SZM incorporates digital formalization and algorithms recognizable by the scientific community, overcoming the

“lower status” view attributed to analog, but without giving up its practical intuition.

- **Restricted applicability:** by merging continuous and discrete processing, the SZM broadens the scope of application, encompassing both continuous variables (physical and biological phenomena) and discrete data (symbolic information and logical control).
- **Technological obsolescence:** the SZM is conceived as a reconfigurable and expandable architecture, which can be implemented on modern platforms (FPGAs, GPUs, programmable analog circuits), ensuring currency and technological scalability.

In this way, the SZM not only recovers the historical advantages of analog computation, but projects them into a contemporary context of integration with digital technologies, creating a hybrid architecture capable of confronting complex problems with robustness, efficiency, and adaptability.

X.20.3 Analog Thinking

Chapter 7 of MacLennan (2007)²⁰ addresses the cognitive implications of analog computation, highlighting aspects forgotten in the twentieth-century debates between analog and digital.

- **Intuitive understanding:** analog computers allowed analytically intractable dynamical systems to be understood, exploring parameters in real time. In contrast, digital ones were slower and not very interactive (until recent advances).
- **Know-how vs. know-that:** emphasis is placed on the distinction between procedural knowledge (engineering) and factual knowledge (science). Analog computation reinforced the former, allowing the transfer of practical intuition between systems.
- **Precision vs. accuracy:** digital computers provide results with many decimal places, but this does not guarantee accuracy. Analog ones, with modest precision, induced a more skeptical and realistic posture in the face of uncertainties.

²⁰ The numbering follows the body of the report, where chapter 6 is *Analog Computation and the Turing Limit* and chapter 7 is *Analog Thinking*. The report’s own outline (Section 1, *Report Outline*) omits the Turing-limit chapter and rennumbers *Analog Thinking* as 6 — a divergence internal to the source, not in the text cited here.

Thus, analog thinking values intuitive understanding, interactive exploration, practical knowledge, and a healthy skepticism in the face of excessively precise numbers.

Turing questions

In subsection 6.7, entitled *Transcending Turing Computability*, MacLennan discusses the relevance of analog computation in the face of the formal limits of the Turing Machine.

- Asking whether analog computers go “beyond Turing” requires working within the Church–Turing (CT) framework.
- These models depend on assumptions (arbitrary reals, discretization of time) that influence the answers, but are not central to the practice of analog computation.
- Excessive focus on these formal questions may divert attention from what really matters: robustness, flexibility, and adaptability in real situations.

In the face of these limitations of the CT framework, MacLennan proposes evaluating analog computation from **frames of relevance** oriented to practice, privileging robustness, adaptability, and experimental efficiency.

Frames of relevance in analog computation

The concept of *frames of relevance*, introduced by MacLennan (2004), refers to the context in which we evaluate the value of a computational technology. In the case of analog computation, this concept shifts the focus from formal discussions (such as the limits of Turing) to scientific and practical criteria.

Definition

A frame of relevance is the set of criteria used to judge the importance and usefulness of a technology. For analog computation, MacLennan points to the following as the main criteria:

- **Robustness:** operating reliably under noise, measurement errors, and physical imprecisions.
- **Flexibility:** modeling continuous, nonlinear, and highly complex phenomena.
- **Interactive exploration:** adjusting parameters in real time and observing the dynamic behavior.

- **Practical efficiency:** providing useful solutions to concrete problems, such as multiscale systems, stiff ODEs, complex PDEs, chaotic dynamics, and massive biological networks.

Contrast with the Church–Turing framework

While the **Church–Turing (CT) framework** defines computability in ideal terms — asking whether something is computable in principle —, frames of relevance reformulate the question as: *is it useful and feasible to compute this in the real physical world?*

This change of perspective helps to understand the role of analog computers: they should not be judged solely through the formal lens of CT, but also by their practical contribution to science and engineering.

Implications for the SZM

In the context of the SZM, frames of relevance provide clear guidelines:

- The SZM does not seek legitimacy through formal proofs of being “beyond Turing,” but rather through its **usefulness in solving hard problems**.
- Its hybrid analog-digital architecture is designed to be robust and adaptable, in tune with practical frames of relevance.
- The emphasis is on the **interactive and hierarchical exploration** of dynamical systems, reinforcing the advantages of analog computation.

X.20.4 Beyond Turing and the SZM

This section compares the main “beyond Turing” proposals in analog computation and relates each of them to the SZM, as summarized in Table 15.

The “beyond Turing” results depend, to a large extent, on idealizations (arbitrary reals, absence of noise, exact comparisons). The SZM adopts another route: **practical gains** in difficult continuous and nonlinear regimes (chaotic systems, multiscale dynamics, PDEs with exotic nonlinearities, biological networks) by means of a **hybrid architecture, a hierarchy of zooms, and directed use of the continuous**, without clinging to formal super-Turing credentials. This view is aligned with Chapter 6: *the frame of relevance* of analog computation privileges robustness, adaptability, and interactive parameter exploration.

Table 15 – Analog-computation approaches and the Turing question, with implications for the SZM.

Approach/ Author	Base model	Claim (beyond Turing)	Hypotheses/ idealizations	Implications for the SZM
Siegelmann and Sontag (1994), Siegelmann (1999)	Recurrent neural networks (RNN) with real weights	Certain continuous RNNs can compute beyond Turing (super-Turing)	Real weights/representations of arbitrary precision; absence of limiting noise; ideal comparators/thresholds	The SZM can exploit continuity and physical nonlinearities without requiring “exact real” weights; focus on robustness and hybrid analog-digital architecture
Rubel (1988), Rubel (1993)	GPAC and EAC (Extended Analog Computer) based on ODEs/PDEs	The EAC solves classes of problems that exceed the Turing scope (in certain regimes)	Exact continuous operators; ideal components for ODEs/PDEs; without unwanted quantization	SZM as an <i>embodied EAC</i> : a reconfigurable physical device that accepts <i>stiff</i> regimes and difficult PDEs, prioritizing stability/practicality
Pour-El and Richards (1979), Pour-El and Richards (1981), Pour-El and Richards (1982)	Analysis of ODEs/PDEs with computable initial data	There exist equations (e.g., wave) with non-computable solution	Exact mathematical idealization; absence of noise/measurement errors	Reinforces that continuous physical systems can induce trajectories not reducible to the digital; the SZM exploits continuous dynamics with hierarchical <i>zooms</i>
Wolpert (1991), Wolpert and MacLennan (1993)	Linear field computers	Universality (Turing) and conceptual extensions via fields	Ideal field medium, controllable linearity, perfect readout	SZM as a nonlinear and hierarchical <i>field computer</i> ; it privileges scalability and integration with control and measurement
MacLennan (2007) (Chapter 6, 6.7)	Frames of relevance for analog computation	Surpassing Turing is a question of the CT framework; it requires robustness, adaptability, flexibility	Evaluation focused on usefulness under noise, uncertainty, and cost	Guideline for the SZM: prioritize robust performance on real problems (stiff ODEs/PDEs, control, biology, etc.), not formal super-Turing proofs

Note: the Successive Zooms Method is part of a broader context of the resurgence of analog and hybrid computation as an architectural alternative for problems of high computational cost. Appendix Y develops this panorama, discussing the acceleration of Partial Differential Equations (PDEs) in High-Performance Computing (HPC) environments, the hybrid analog-digital paradigm, electrical and photonic architectures, and an original proposal for frequency-based analog computation — the *Frequency-based Programmable Analog Computer* (FPAC). The relationship between these architectures and the SZM is developed in detail in the corresponding sections. An application of the SZM to the discovery of nonlinearities in fundamental fields of particle physics, coupled with the SINDy method, is developed in Appendix S (Vol. II), Section S.3 (p. 350).